



UNIVERSITY OF
GEORGIA
College of Engineering

ECSE 2920

ECSE Design Methodology

Sound Tracking Autonomous Rover (S.T.A.R.)

Technical Documentation

Aryan Patil
Derrick Cannon
Jay Bohmer
Justin Moreno
Miguel Soldevilla

April 23rd, 2025

Table of Contents

<i>Section 1 – Objectives.....</i>	<i>3</i>
<i>Project Objectives</i>	<i>3</i>
<i>Section 2 – Constraints.....</i>	<i>4</i>
<i>Software Constraints.....</i>	<i>4</i>
<i>Hardware Constraints.....</i>	<i>4</i>
<i>Section 3 – Software Design.....</i>	<i>6</i>
<i>3.1 General Software Aspects</i>	<i>6</i>
<i>3.2 Dip Switch States.....</i>	<i>7</i>
<i>3.3 Main Code Loop</i>	<i>8</i>
<i>3.4 Signal Processing</i>	<i>9</i>
<i>Section 4 – Hardware Design</i>	<i>10</i>
<i>4.1 – Raspberry Pi Protection Hat.....</i>	<i>10</i>
<i>4.2 – Motors and H Bridge.....</i>	<i>11</i>
<i>4.3 – Rotary Encoders</i>	<i>14</i>
<i>4.4 – DIP Switch</i>	<i>17</i>
<i>4.5 – Push/Pull Amplifier.....</i>	<i>18</i>
<i>4.6 – Analog Frequency Filter</i>	<i>23</i>
<i>4.7 – Half Wave Rectifier.....</i>	<i>26</i>
<i>4.8 – Analog to Digital Converter (ADC)</i>	<i>28</i>
<i>4.9 – Bump Switch.....</i>	<i>34</i>
<i>Section 5 – Printed Circuit Board (PCB) Design</i>	<i>36</i>
<i>5.1 – CAD Software Overview</i>	<i>36</i>
<i>5.2 – CAD Schematic Design</i>	<i>37</i>
<i>5.3 – Component Placements</i>	<i>38</i>
<i>5.4 – Preparation for Manufacturing.....</i>	<i>40</i>
<i>Section 6 – Ethics.....</i>	<i>42</i>
<i>Section 7 – Self Critique.....</i>	<i>44</i>
<i>Section 8 – Revision Plan.....</i>	<i>46</i>
<i>Section 9 – Appendix.....</i>	<i>47</i>
<i>References</i>	<i>62</i>

Section 1 – Objectives

The objective of this project was to design a car which can autonomously identify, track, and move to a fixed set of audio frequencies.

Project Objectives

This project had 2 primary objectives, which are in order as follows:

1. Design a car that can move in:
 - a. A straight line for 3 feet.
 - b. A square pattern with each side measuring 3 feet, for a total of 2 times.
 - c. A pattern that resembles the figure “8”, for a total of 2 times.
2. Design a car that can identify, track and move towards a sound source placed 3 – 5 feet away from the car emitting a frequency of 4186 Hz, also known as the C8 frequency.

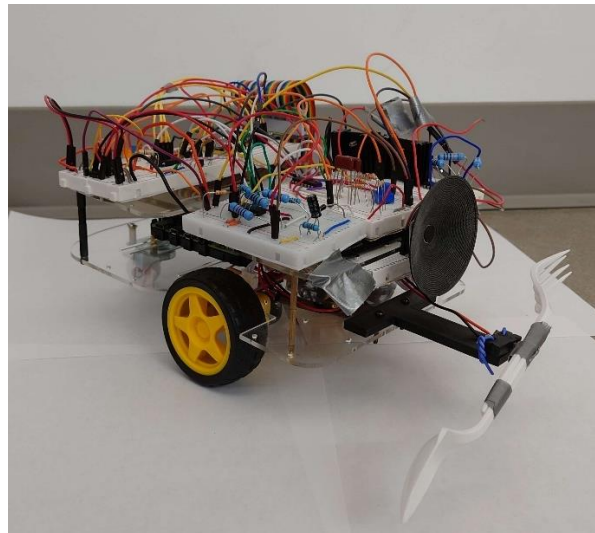


Image 1.1: Completed Car Design.

Section 2 – Constraints

Like any other project, the design and implementation of this car had well-specified software and hardware constraints. Most of these constraints are in place to minimize costs and to implement designs in an easy but efficient manner.

Software Constraints

This project has several software constraints, most of which are related to the onboard Single-Board Computer (SBC) which in this case is the Raspberry Pi (RPI) 4 Model B.

- The main code must be written in *Python*, preferably Python 3.
- The default operating system on the RPI, Raspberry Pi OS, is based on Debian Linux and any required packages and libraries must be compatible with Debian Linux.
- Geany IDE is the default GUI text editor on the RPI. Alternate GUI text editors may be used on the RPI, provided they are compatible with Debian Linux.
- Git can be used to push/pull commits from the team's repository on the RPI. Alternatively, a SSH terminal can also be set up for faster deployment and testing of code.
- The RPI can only read digital data, an Analog to Digital Converter (ADC) must be used to process analog data.

Hardware Constraints

This project has several hardware constraints, most of which are rooted in the limitations of the cost, power architecture, and physical design.

- Designers may refer to the table of components in [Appendix 2 – Hardware Constraints](#) to find the list of items used for the actual design of this car.
- The Raspberry Pi 4 Model B has 26 user-programmable digital GPIO pins and 2 PWM (Pulse Modulated Wave) pins. The RPI does not feature any analog GPIO pins.
- The car is powered by two distinct power sources: a 24 V DC power supply for analog circuit components, and a 5 V DC power supply for the Raspberry Pi.
- To enable proper operation of analog circuits like the analog frequency filter, a virtual ground should be created midway between 24V and 0V (+ 12 V, - 12 V).

- The virtual ground is shared across both the 24V supply and the Raspberry Pi's dedicated 5V supply, ensuring compatibility between analog and digital domains without voltage mismatches.
- Decoupling and bypass capacitors are placed strategically near sensitive components to reduce ripple, suppress transients, and isolate noisy loads like motors.
- Designers may use
- This car requires the use of an H-Bridge to control the DC motors. The L293 IC was used for the design of this car.
- The DC motors must be stepped down to 6 V using a buck converter.
- This car requires the use of an analog filter to detect the C8 sound source. A quad OP-AMP can be used to design the filter. The TL-084 was used for the design of this car.
- This car requires the implementation of an Analog-to-Digital Converter (ADC). The LM-339 comparator was used to implement the ADC for digitizing the filter output for the Raspberry Pi.
- All wiring and connectors need to be robust enough to withstand physical movement and vibration during vehicle operation. Secure mounting, strain relief, and careful cable routing should be used to reduce the risk of disconnection.

Section 3 – Software Design

Software is an integral part of the S.T.A.R. project. The Raspberry Pi serves as the “brain” of the whole project, deciding what to do with the input signals it receives and what to control by outputting different signals. All code used on the S.T.A.R. can be found on Team 5’s [GitHub repository](#).

3.1 General Software Aspects

Objective

The Raspberry Pi runs on its own dedicated Linux-based Operating System. Python is the coding language used for the S.T.A.R. due to its versatility with libraries and simplicity to start coding. The team used one main library for controlling the GPIO pins through code, pigpio.

Raspberry Pi OS and terminal

The Raspberry Pi uses an operating system specifically designed for it, but the base of the Raspberry Pi OS is Linux. Linux is a terminal heavy operating system, meaning that many important functions must be executed by writing commands in a terminal. One such important function is automatically executing the Python code when the Raspberry Pi boots. A feature called crontab exists specifically for running commands at time intervals, including each time the system boots. The crontab itself is a script file that contains the list of commands to execute at designated times. To run the main python file, a small bash script containing the terminal commands that would be executed is opened and ran on reboot.

Pigpio Library

On its own, Python cannot interact with the Raspberry Pi’s GPIO pins. An external library is required to interface with each pin. The library chosen for the S.T.A.R. is pigpio, which is a powerful module with many features including but not limited to on the [pigpio website](#).

- Reading/writing GPIO pins and setting their modes
- Callbacks when any GPIO 0-31 change state
- Hardware timed PWM on any GPIO 0-3

3.2 DIP Switch States

Objective

The objective of the Dip Switch States is to provide an indication of what mode of operations is active during runtime and allow the user to select different modes of operation on the S.T.A.R.

DIP States

The Dip switch configurations are set according to the requirements provided on GitHub. The states determine the mode of operation the S.T.A.R will execute when the bit number is flipped on.

Bit 3 which is read by GPIO 17 is the “GO” bit. If the GPIO pin reads a low state then the car remains in a “Don’t Move” state, regardless of any other button configuration. When Bit 3 is logic high then the code will evaluate the remaining pins which correspond to the remaining bits.

The software handles logic by using a function named *inputCases()*. This function reads the GPIO inputs and returns a value 0-6 that represents the different modes of operation.

- Input Cases returns 0 for the mode of operation *Don’t Move*
- Input Cases returns 1 for the mode of operation *Single sound source*
- Input Cases returns 3 for the mode of operation *Straight*
- Input Cases returns 4 for the mode of operation *Figure eight*
- Input Cases returns 5 for the mode of operation *Square*
- Input Cases returns 6 for the mode of operation *Debug*

A complete table of all configurations and the corresponding input values is available in [Appendix Table 3.2.1.](#)

In the main program loop, the function *inputCases()* is repeatedly called. Based on the return value, an if statement is used to check the value *inputCases()* returns and the appropriate mode of operation can execute.

To indicate the current dip switch state each bit is mapped to an LED by setting four GPIO pins to output and setting the output pins to high or low to represent the bit. A complete table of the GPIO pins that are used to represent the Bit states are shown in [Appendix Figure 3.2.2.](#)

3.3 Main Code Loop

Objective

The code works by running in a main while true loop with different cases, then executes each specific function defined. Some functions are written in separate classes to ease integration in the main file. Other functions are written within the main file, but they are written as separate functions to drastically reduce the size of the main while loop. Separate functions in the main while loop also allow it to be easier to follow.

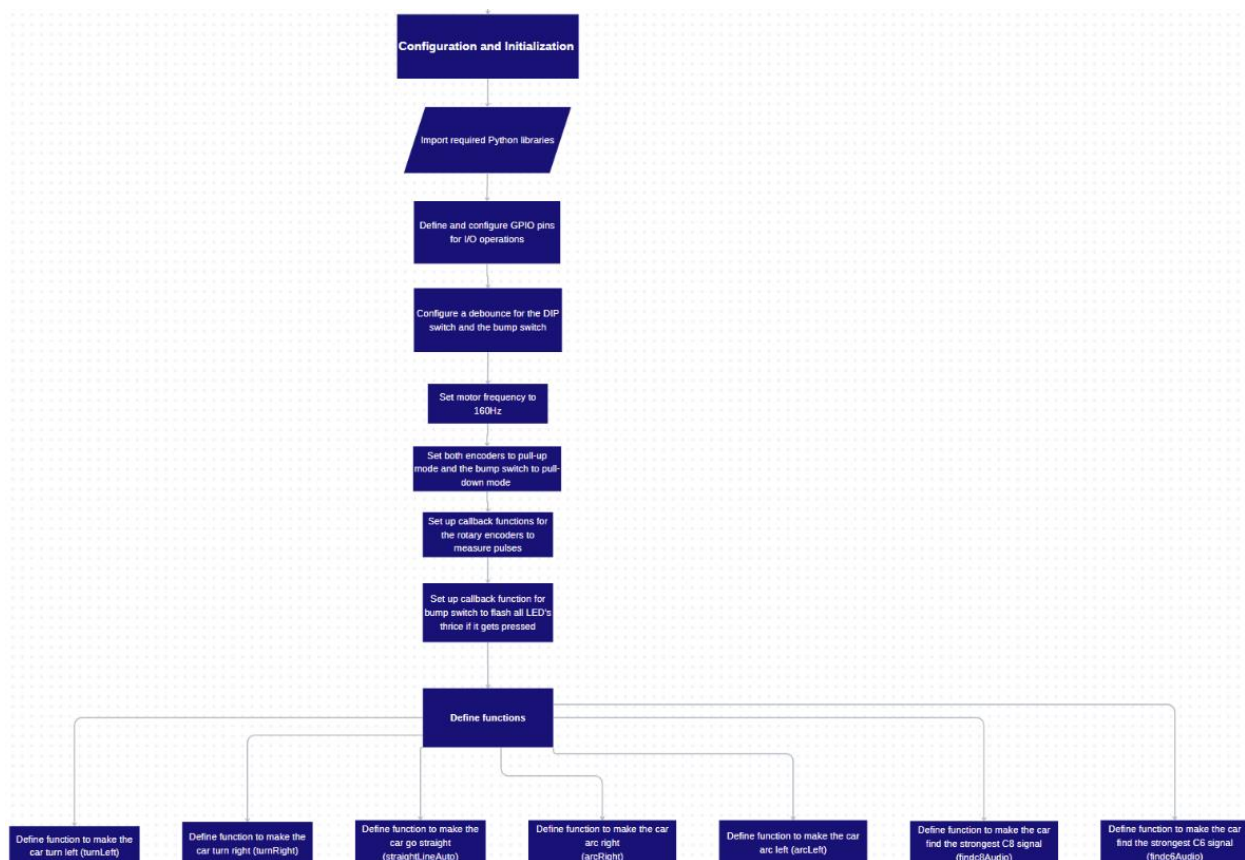


Figure 3.3.1 Code Block Diagram for the main code loop

While True Loop

The main while true loop works by waiting for the enable switch to be flipped, then detects which case is active and executes the desired functions. Each individual function is detailed step by step within code comments, but an important distinction is the bump switch callback. A callback function is used for whenever the bump switch is triggered, which then overwrites the while loop to stop any function active, and blinks every light multiple times.

3.4 Signal Processing

Objective

Each GPIO pin is designated a specific functionality to be used throughout the project. They can be either inputs, outputs, PWM, or predefined such as power or ground. A GPIO diagram was used throughout the project to keep track of what GPIO pin does what and what configuration it is in.

Pull Up/Pull Down	Used State	Default State	PIN Function	PIN Label	PIN #	PIN #	PIN Label	PIN Function	Default State	Used State	Pull Up/Pull Down
				3.3V PWR	1	2	5V PWR				
				GPIO 2	3	4	5V PWR				
				GPIO 3	5	6	GND				
				GPIO 4	7	8	UART0 TX				
				GND	9	10	UART0 RX				
DOWN	INPUT	INPUT	ADC Comp High	GPIO 17	11	12	GPIO 18	PWM0/Left Motor EN signal	LOW	PWM0	
DOWN	INPUT	INPUT	Dip Switch Input Bit 0	GPIO 27	13	14	GND	Common Ground			
DOWN	INPUT	INPUT	Dip Switch Input Bit 1	GPIO 22	15	16	GPIO 23	Dip Switch Output 3	LOW	OUTPUT	DOWN
DOWN	INPUT	INPUT	Dip Switch Input Bit 2	GPIO 10	17	18	GPIO 24	Dip Switch Output 2	LOW	OUTPUT	DOWN
				3.3V PWR	19	20	GND	Common Ground			
				GPIO 9	21	22	GPIO 25	Dip Switch Output 1	LOW	OUTPUT	DOWN
				GPIO 11	23	24	GPIO 8				
				GND	25	26	GPIO 7				
				Reserved	27	28	Reserved				
				GPIO 5	29	30	GND				
UP	INPUT	INPUT	Left Optical Switch Read	GPIO 6	31	32	GPIO 12	ADC Comp Low			
UP	INPUT	INPUT	Right Optical Switch Read	GPIO 13	33	34	GND	Common Ground			
N/A	PWM1	LOW	PWM1/Right Motor EN signal	GPIO 19	35	36	GPIO 16	ADC Comp Med			
DOWN	OUTPUT	LOW	Dip Switch Output bit 0	GPIO 26	37	38	GPIO 20	Bump Switch Enable	LOW	INPUT	DOWN
				GND	39	40	GPIO 21				

Figure 3.4.1 GPIO diagram used throughout the project

GPIO Pin Functionality

Each GPIO pin at the beginning of the code must be designated the following: default state, input or output, and pull-up or pull-down configuration. The default state is only used for outputs, as it determines the threshold. Input and output state is used for whether the pi will control voltage or if an outside voltage will control the pin. Any input signal must be above a certain voltage threshold to register as a logical high. The pull-up and pull-down state is used to keep a reference voltage. For example, the dip switch bits are open circuits when not active, so the pull-down configuration makes the voltage pull-down to ground reference, making it a logical low.

Section 4 – Hardware Design

Functional hardware is an important component of any physical electrical product. This project uses several key pieces of circuitry to accomplish its primary objectives, and they are detailed about in this section.

4.1 – Raspberry Pi Protection Hat

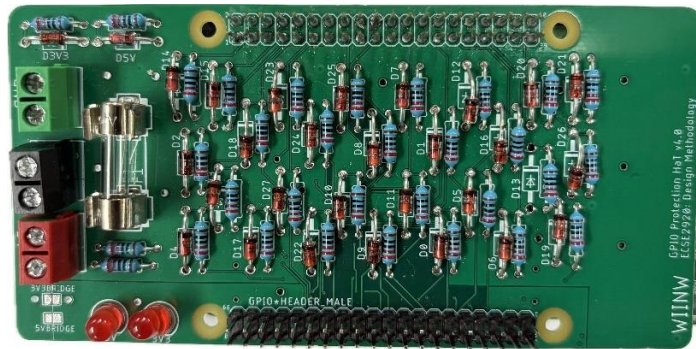


Image 4.1.1: Raspberry Pi Protection Hat

The Raspberry Pi Protection Hat is a circuit and hardware module that protects the Raspberry Pi from damage due to excessive currents and voltages.

1. The fuse prevents the connected circuit from drawing too much current from the 3.3 V and 5 V GPIO pins, therefore preventing them from “frying themselves”.
2. The resistors regulate current flow to GPIO pins on the RPI.
3. The Zener diodes function as clamping circuits as they redirect any signals to ground if they are above the threshold voltage (3.3 V) for the GPIO pins on the RPI.

Some suggestions for using the Raspberry Pi Protection Hat in this project are –

1. It is a bad idea to use the RPI without the protection hat. It is easier and cheaper to replace a 5-dollar fuse than a 70-dollar Raspberry Pi.
2. It is a good idea to invest in a few fuses before starting the project. A fuse may melt at any given moment for a multitude of reasons and halt any further progress. Find the reason why the fuse was blown, find a solution to the problem and implement it, replace the fuse, and continue testing with care.
3. Do not hot plug the Raspberry Pi Protection Hat. Voltage surges from suddenly connecting components can blow the fuse on the protection hat.

4.2 – Motors and H Bridge

Objective

The objective of this section is to integrate two DC motors into the S.T.A.R. platform and control their direction and speed using the Raspberry Pi 4 and an H-bridge driver. This is accomplished using the L293 Quadruple Half-H Driver, which allows the Pi to command the motors through GPIO signals and Pulse Width Modulation (PWM). Motor actuation is essential to S.T.A.R.'s ability to navigate toward sound sources and perform its core function as an autonomous, audio-responsive vehicle.

To achieve precise motor control, the system must not only manage directional logic but also vary the motors' speed in response to changing conditions. This required a complete understanding of the H-bridge's operation, careful mapping of GPIO pinouts, and reliable integration with the Raspberry Pi's control logic.

Understanding DC Motors and H-Bridge Functionality

Before working with the Raspberry Pi, the team needed to first develop a solid understanding of how DC motors and H-bridge motor drivers function together. A DC motor spins forward when the current flows in one direction and reverses when the current direction is flipped. This made it essential to use a circuit capable of reversing the polarity of the applied voltage without physically swapping wires.

This is where the H-bridge circuit comes into play. An H-bridge is a four-switch circuit that allows current to flow in either direction through a load, in this case, the motor. By activating different combinations of these switches, the polarity of the voltage across the motor can be reversed, allowing it to spin clockwise, counterclockwise, brake, or coast.

The L293 motor driver IC integrates two full H-bridge circuits into a single package. As shown in Figure 4.3.1, each channel includes:

- **Two input pins** that determine the direction of current flow.
- **Two output pins** that connect directly to the motor.
- **An enable pin** that acts as a master switch for the channel. If the enable pin is LOW, the motor is disabled regardless of the input states. If the enable pin is HIGH, the motor operates according to the inputs.

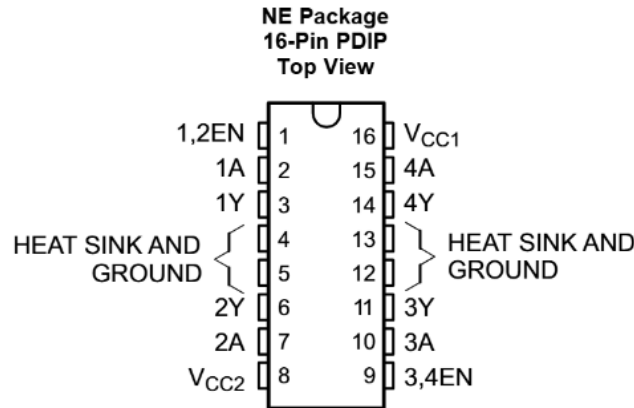


Figure 4.2.1: L293 IC Pinout.

Initial testing was performed using a function generator to simulate digital inputs at the H-bridge input pins. By applying different HIGH and LOW combinations, the team was able to observe:

- IN1 = HIGH, IN2 = LOW → motor spins forward
- IN1 = LOW, IN2 = HIGH → motor spins in reverse
- IN1 = IN2 (both HIGH or both LOW) → motor brakes or coasts

In these tests, both motors behaved as expected, and the team observed that toggling the enable pin acted like a power switch, disabling or enabling motor function entirely. This revealed that speed could be controlled by applying a PWM signal to the enable pin, while direction could be controlled via fixed logic states on the input pins.

Motor Connection and Power Configuration

While the L293 H-bridge driver is capable of full bidirectional control using its dual-input design, in this version of STAR the motors were configured for forward-only motion to simplify wiring and reduce software complexity during early development. A future revision of the system may explore utilizing the H-bridge's full capability to enable both forward and reverse motion through dynamic GPIO control of the input pins (see Section 8: Revisions).

For the initial configuration, the direction control inputs were hardwired rather than GPIO-controlled:

- Inputs 1A (left motor) and 4A (right motor) were hardwired to 3.3V, keeping them permanently HIGH.
- Inputs 2A (left motor) and 3A (right motor) were hardwired to GND, keeping them permanently LOW.

This wiring setup ensured that both motors would always spin in the forward direction when enabled. Rather than toggling inputs for direction, the team used only the enable pins to control when each motor was active and at what speed.

- The right motor's enable pin (EN1) was connected to [GPIO 13 \(PWM1\)](#).
- The left motor's enable pin (EN2) was connected to [GPIO 18 \(PWM0\)](#).

This can be seen in the GPIO Diagram in Appendix 3.4.

These pins provided PWM signals from the Raspberry Pi, allowing the team to modulate motor speed by adjusting the duty cycle in software. When the PWM signal was HIGH, the motor received power; when LOW, it was disabled. This approach provided effective speed control with minimal logic complexity.

For power distribution, the L293 features two separate voltage inputs:

- VCC1, powering the internal logic of the H-bridge, was connected to the 5V rail from the Raspberry Pi.
- VCC2, powering the motors, was supplied by a buck converter that stepped down 12V to 6V—a safe and appropriate voltage for the TT DC Gearbox Motors.

This separation of logic and motor power helped protect the Raspberry Pi from electrical noise and current spikes generated during motor operation, while the buck converter ensured consistent and reliable performance under load. The final configuration offered a stable and safe solution for unidirectional motion during early testing and encoder-based control development.

4.3 – Rotary Encoders

Objective

The objective of this subsystem is to accurately measure the rotational speed (RPM) of STAR's drive motors using optical feedback. By gathering real-time speed data, the system enables performance calibration, motor response testing, and the derivation of transfer functions for precise control. This was achieved using Coded Disc Encoders with 20 holes in conjunction with optical endstop sensors, all mounted around the output shaft of a TT DC Gearbox Motor.

Understanding the Encoder System

The coded disc encoder comprises a small plastic disc affixed to the motor's shaft, punctured with 20 evenly spaced holes around its circumference. An optical endstop (IR emitter/detector pair) is positioned so that, as the motor spins, the detector sees a rapid sequence of light pulses whenever a hole passes between the emitter and detector. Each pulse corresponds to one twentieth of a revolution; by counting pulses per unit time, the system computes RPM with the formula:

$$RPM = \frac{\text{Pulses in Interval}}{\text{Holes per Revolution}} \times \frac{60}{\text{Interval (s)}}$$

Physical Integration and Mounting

Each encoder system was mechanically mounted to the rear shaft of the motor. The optical endstop sensors were screwed to the topside of the bottom chassis, with the sensors protruding slightly into the interior. The Coded Disc Encoder sits between the emitter and detector arms of the endstop, precisely aligned with the beam path. The sensors were mounted from the top of the chassis, peeking into the underside, and carefully spaced to ensure that the disc passes cleanly through the IR beam without friction or misalignment.

This physical arrangement allowed for consistent pulse generation while minimizing mechanical wear or interference with other components. The encoders were aligned so that the disc remained parallel to the sensors, preserving accuracy during full-speed motor operation.

Encoder Wiring and GPIO Configuration

To process the pulses generated by the encoder discs, each optical endstop was connected directly to the Raspberry Pi's GPIO system. Each sensor features a standard three-pin interface: VCC (power), GND (ground), and Signal (SIG). These connections allow the Raspberry Pi to detect rising digital pulses whenever a hole in the rotating encoder disc passes through the IR beam.

The encoders are configured using the following GPIO pins.

- **GPIO Pin 5** (Physical Pin 29) for left encoder signal
- **GPIO Pin 6** (Physical Pin 31) for right encoder signal
- **3.3V rail** (Physical Pin 1) for power
- **Ground** (Physical Pin 6) for reference

This setup enables the Raspberry Pi to independently monitor pulse signals from each motor's encoder. These signals are later used to determine motor speed and total rotation counts. The software implementation, including GPIO initialization and pulse counting logic, is detailed in Section 3.4.

Transfer Function Derivation and Motor Calibration

To characterize the behavior of each motor, RPM data was collected at various PWM duty cycles using the optical encoder system. For each duty cycle tested, the number of encoder pulses over a fixed time interval was recorded and converted into RPM. This data was then plotted for both motors, with duty cycle on the x-axis and RPM on the y-axis.

The resulting graphs, shown in Figures 4.3.1 and 4.3.2, display a clear linear relationship between PWM input and motor speed for both the left and right motors. A line of best fit was applied to each data set using linear regression, yielding equations of the form:

$$RPM = m * (\text{Duty Cycle}) + b$$

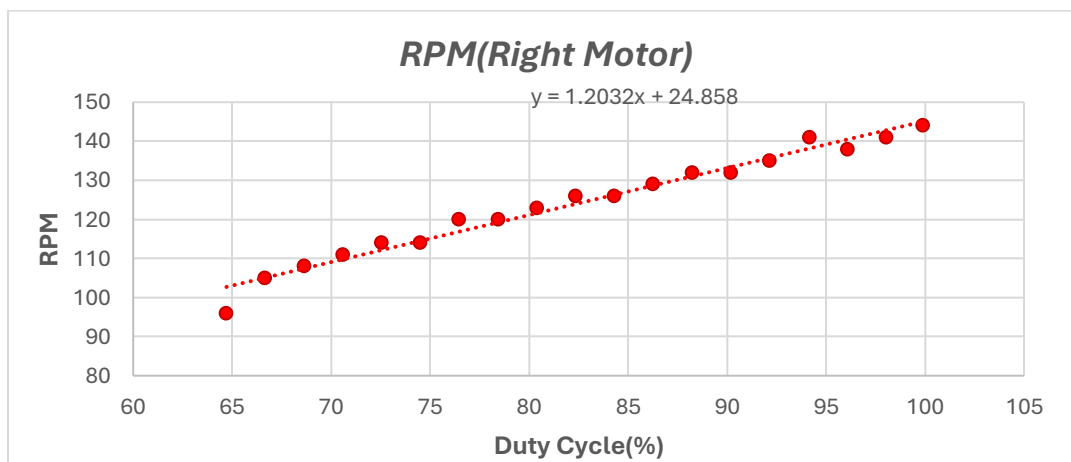


Figure 4.3.1: Transfer Function for the Right Motor

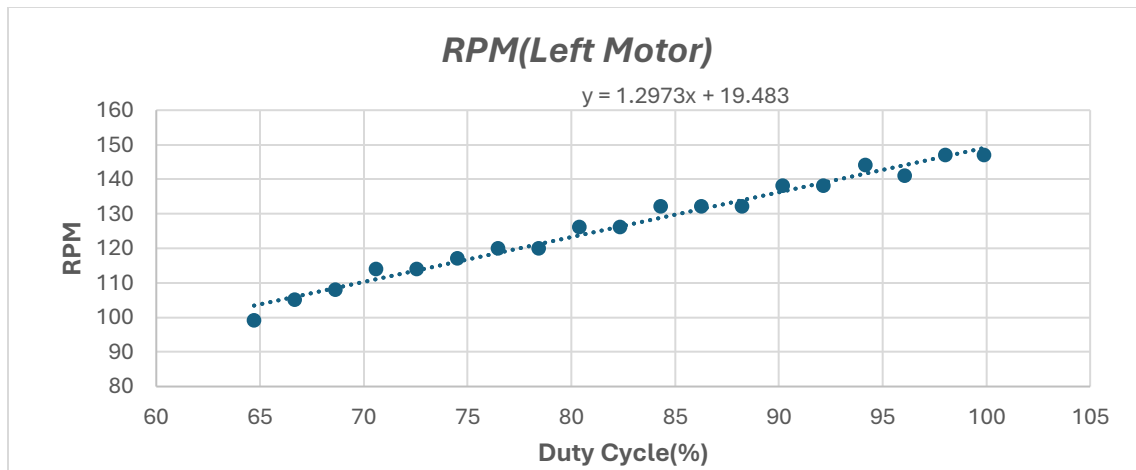


Figure 4.3.2: Transfer Function for the Left Motor

These equations represent the static transfer functions of the motors, where the slope m indicates the motor's speed sensitivity to PWM, and the intercept b accounts for any dead-zone or offset in the motor's response.

Upon comparing the slopes of the two lines, it was evident that the motors did not respond identically. Minor discrepancies in internal motor construction or gear resistance caused one motor to consistently run faster than the other at the same duty cycle. To address this, a scaling coefficient was calculated by dividing the slopes of the two lines. This coefficient was then used in software to scale the PWM signal sent to the faster motor, ensuring both motors rotated at the same effective speed.

This calibration step allowed S.T.A.R. to maintain straight-line motion during open-loop operation, where motor speed was controlled based on pre-measured PWM-to-RPM relationships without using real-time feedback. This process laid the foundation for the system's later transition to closed-loop control using encoder pulse counting (see Team 5's [GitHub repository](#)).

4.4 – DIP Switch

Objective

The objective of the DIP switch is to allow manual mode selection for the S.T.A.R. Different combinations of switches should represent different modes of operation.

Dip Switch

A DIP switch works as a collection of small switches that can be manually toggled. The dip switch provided is an 8-pin dip switch which includes 8 individual switches which correspond to their own bits when implemented in software. When a switch is toggled on, it essentially completes an electrical circuit allowing current to flow, so when off the it represents an open circuit.

For the implementation of the DIP switch a 3.3V provided by the RP4 is connected to GPIO pins 1, 2, 6, and 7. On the other side of the DIP switch each pin will be connected to a GPIO pin that is set to an input that can read when the switch is flipped completing the circuit and reading a High logic level which is provided by the 3.3V. When the switch is off the circuit is essentially open, so it is left at a floating state neither high nor low leading to unpredictable behavior when reading the DIP switch. To resolve the issue of floating state, a pull-down resistor of 10 k Ω is added to each pin to ensure that the GPIO pin can read a low value when the switch is open. A labeled schematic of the DIP switch wiring is included in [Appendix 4.5.1](#).

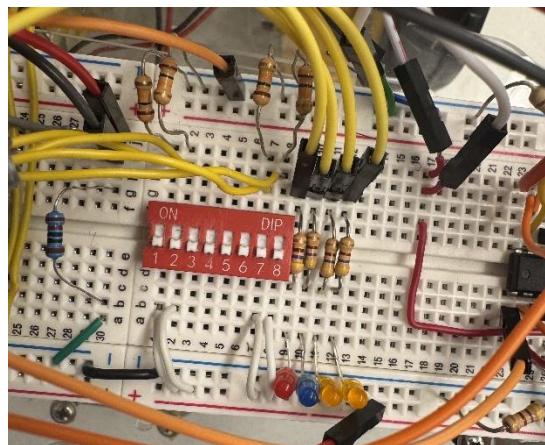


Figure 4.4.1: Hardware implementation of DIP Switch

4.5 – Push/Pull Amplifier

Objective

The objective of designing and creating a Push-Pull amplifier circuit is to initialize a reference point from creating $\pm 12\text{V}$ split from the 24V DC power supply. This reference point is referred to as virtual ground and proves critical to components that require power from the 24V DC power supply.

Purpose of the Push/Pull Amplifier Circuit

The function of a Push-Pull amplifier circuit is to control current flow using active components. These components alternately "push" and "pull" current depending on the load connected at the circuit's output. This circuit is designed to manage both positive and negative voltage inputs by establishing a reference point with the process of splitting a single 24 V DC power supply into $\pm 12\text{V}$. This reference point serves as a ground point in this circuit and is referred to as "virtual ground". Virtual ground plays a critical role in providing a stable reference voltage necessary for powering operational amplifiers used in filter circuits and Analog-to-Digital Converters (ADCs).

Designing a Digital Push/Pull Amplifier

As depicted in [Appendix Table 2.1.1 - Components Used for the Design of the Car](#), the provided power supply is a single 24V DC source rather than a dual-polarity supply. Therefore, as previously discussed in the [Purpose of Push/Pull Amplifier Circuit](#) section, establishing a midpoint reference (virtual ground) is necessary to split the 24V DC source into $\pm 12\text{V}$. This split is achieved by applying the voltage division principle, as shown in [Appendix Figure 4.5.1 - Power Supply Voltage Divider](#), where two equivalent resistors are connected in series. [Appendix Section 4.5](#) mathematically illustrates how two equivalent resistors create an even split of the 24V DC supply.

However, this simple voltage divider alone does not achieve the required $\pm 12\text{ V}$ split without introducing a virtual ground as the midpoint reference. As illustrated in [Appendix Figure 4.5.1](#), ground is set at the negative input of the 24V source. This results in incorrect node voltages achieved. The top node measures +24V, the midpoint measures +12V, and the bottom node measures 0V. When a virtual ground is incorporated at the midpoint, as shown in [Appendix Figure 4.5.2](#), the voltages correctly measure +12V at the top node and -12V at the bottom node, both referenced to the virtual ground. Thus, the virtual ground serves effectively as the reference ground specifically for splitting the 24V DC power supply into dual polarities.

With the successful splitting of the 24V DC source into $\pm 12\text{V}$, the Push-Pull amplifier circuit can now be designed under the hardware constraints of using a P-channel MOSFET and an N-channel MOSFET, as specified in [Appendix Table 2.1.1](#). Utilizing this split voltage, the circuit must be capable of both pushing and pulling current, as the name implies. As indicated in the [References](#) section of the appendix, research into push-pull amplifier circuits utilizing both types of MOSFETs led to the initial iteration of the circuit design shown in [Figure 4.5.3](#) of the appendix.

Initial Circuit Design Iteration

The initial circuit iteration includes various components integrated with the MOSFETs and power supply, including diodes, capacitors, and resistors. As depicted in the schematic, the diodes are placed in series between resistors R1 and R2. These diodes stabilize the virtual ground node during start-up and ensure a smooth transition from passive diode conduction to active MOSFET conduction. The capacitors, placed in parallel, function as voltage regulators by absorbing voltage spikes and irregularities that may occur when a load is connected. Resistor R5 serves the straightforward purpose of maintaining the virtual ground node at the midpoint and helps ensure the circuit responds smoothly to variations caused by the connected load. The remaining resistors, R3 and R4, act as MOSFET protectors by limiting gate-charge current to prevent instantaneous switching caused by sudden voltage changes from load demands. The MOSFETs, N-type and P-type, function as current sources and sinks. The N-type sources current into the virtual-ground node, while the P-type sinks current from it. By varying their conduction in opposite directions, the MOSFETs supply or absorb exactly the amount of current the load demands, keeping the midpoint reference voltage constant.

A load impedance of $240\ \Omega$ was used for the baseline test, producing an approximate current draw of 50 mA. [Appendix Figures 4.5.4 and 4.5.5](#) show a regulated voltage split, where a positive load resulted in + 11.4 V and + 47.6 mA, and a negative load produced – 11.4 V and – 47.6 mA. As the initial circuit iteration performed adequately under baseline conditions, further testing was conducted with load impedances ranging from $47\ \Omega$ to $5600\ \Omega$. The results are documented in [Appendix Table 4.5.6](#). Analysis of the data revealed that the circuit exhibited improved performance with higher impedance loads, particularly $1000\ \Omega$ or greater, while struggling to maintain regulation under lower impedance loads. As load impedance increased, the voltage split gradually stabilized, achieving a regulated $\pm 12\ \text{V}$ between the two power rails. This outcome prompted the need to maintain stable voltage regulation under varying load conditions, leading to the second and final iteration of the Push-Pull amplifier circuit design.

Final Circuit Design Iteration

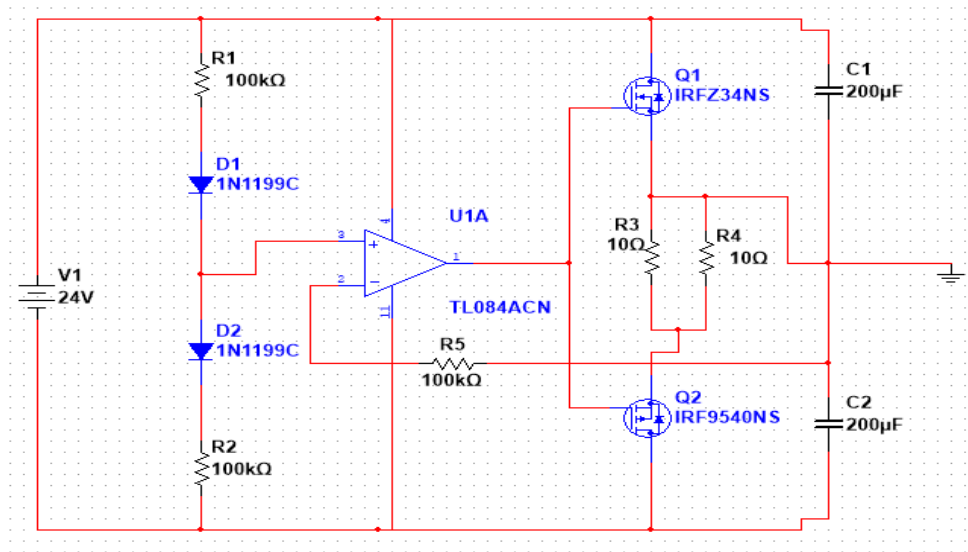


Figure 4.5.7: Final Push-Pull Circuit Iteration Design

Figure 4.5.7 illustrates the initial circuit iteration with the addition of an operational amplifier (op-amp) integrated into the heart of the circuit. The specific op-amp chosen for this configuration is the TL081ACN. An op-amp was selected due to its ability, when combined with negative feedback loop, to adjust its output until the voltages at its inverting and non-inverting inputs become equal. The non-inverting input (pin 3) is connected to the midpoint junction of the R1–R2 resistor divider, establishing the circuit's virtual-ground reference. Although this midpoint reference is considered 0 V for the rest of the system, it sits at +12 V above the negative terminal of the power supply. By continuously adjusting its output, the op-amp stabilizes this virtual-ground node, maintaining the two power rails fixed at ± 12 V relative to it.

Regarding the other pins essential for the op-amp's operation, pin 7 is connected to +12V, and pin 4 is connected to –12V, providing the necessary dual-polarity power supply. The op-amp's output (pin 6) connects to the gates of the P-channel and N-channel MOSFETs, utilizing the op-amp's high voltage gain and negative feedback loop to control the MOSFETs' gate voltages. This configuration determines which MOSFET conducts, based on the conditions of the load.

The inclusion of the op-amp in the circuit results in minor operational adjustments to certain components. Capacitors, MOSFETs, diodes, and resistors R3 and R4 maintain the same roles as described in the initial circuit design iteration. However, resistors R1 and R2 have significantly increased values, greatly reducing their idle current. In the initial circuit, resistors R1 and R2 were crucial for regulating the voltage split, but with the op-amp now introduced, they serve primarily as a reference node, allowing the op-amp to manage voltage regulation.

Additionally, resistor R5 (100 K Ω) now functions as a feedback resistor for the op-amp, ensuring stability and assisting in the op-amp response to variations resulting from load conditions.

Appendix Figures 4.5.8 and 4.5.9 show the final circuit iteration configured for the baseline test of delivering 12 V at 50 mA to a connected load with an impedance of 240 Ω . The test was successful as a positive load connection resulted in a voltage of + 12 V and a current draw of + 50 mA, while a negative load connection produced – 12V and – 50 mA. Following this success, the same load impedance test used in the initial circuit iteration was repeated, with results recorded in **Appendix Table 4.5.10**. The data confirms the effectiveness of the op-amp implementation in regulating the voltage across the power rails, as both remained stable at a consistent $\pm 12\text{V}$ regardless of the load demand.

Physical Implementation

The data gathered from the final digital circuit iteration confirms its ability to maintain a constant voltage split between the two power rails. To verify the circuit in practice, the final iteration was translated into a physical model, requiring construction, testing, and analysis. **Figure 4.5.8** below illustrates the physical implementation of the final circuit design. Using the same load demand test applied in both the first and final circuit iterations, **Appendix Tables 4.5.12 and 4.5.10** were created to document the resulting data. Analysis of this data showed that the translation from digital to physical implementation was successful. The voltage remained regulated at $\pm 12\text{ V}$, with only a slight deviation of approximately 50 mV observed across the range of load connections, from the lowest to the highest impedance.

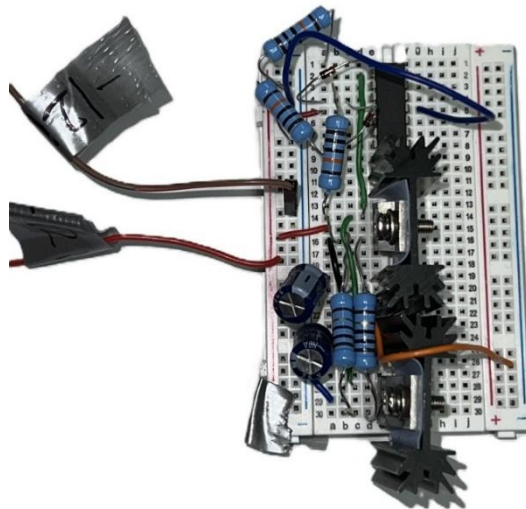


Figure 4.5.8 - Push-Pull Physical Implementation

Designating a common ground

The final testing and analysis, both digital and physical, of the final circuit iteration represents a significant adjustment in how power is supplied to components within the system. The initialization of a virtual ground establishes a new reference point for components such as the op-amp and comparators, which require power from either the +12V or –12V power rails. For reference, the filter design discussed in [Section 4.6](#) utilizes both power rails, while the Analog-to-Digital Converter (ADC), detailed in [Section 4.8](#), operates from the positive rail in reference to virtual ground. The H-Bridge motor driver (L239) also draws from the power supply, with assistance from a buck converter that steps down the voltage.

Given that all these components require power from the Push-Pull amplifier circuit, determining whether a component should be grounded to the RP4 or virtual ground becomes a significant design consideration. The solution involves tying virtual ground to the RP4 ground with the goal of establishing a common ground reference across the system. This is possible because the RP4 ground is considered “floating,” meaning it can shift its reference potential based on the input voltages of connected components.

4.6 – Analog Frequency Filter

Objective

The objective of designing and implementing an analog filter was to create a circuit that can receive an AC signal from the microphone and only let through signals that are of a particular frequency. In this case, the desired frequency was 4186 Hz, which is also known as the C8 frequency. The designer is allowed to have a maximum bandwidth of 1000 Hz.

Choosing a filter design

As the filter is targeting a narrow band of frequencies, a bandpass filter was deemed necessary to filter out the desired frequency. A band pass filter is the combination of a low-pass filter (filter for frequencies lower than the desired frequency) and a high-pass filter (filter for frequencies higher than the desired frequency). Designers have a variety of filters to choose from for this task, but the 2nd Order Multiple Feedback Bandpass (MFB) Filter was deemed to be a suitable option for this project due to its relative design simplicity and support for large Quality Factors (Q).

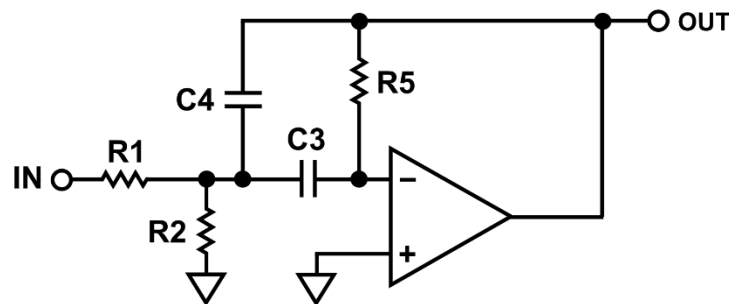


Figure 4.6.1: Multiple Feedback Bandpass Filter Circuit Schematic

Operation of filter

The filter can be understood as a combination of a high-pass and low-pass filter.

- The impedance of a capacitor is inversely proportional to the frequency of the AC signal.
- At low frequencies, the impedance of C3 is very high, therefore it blocks those low frequencies. Therefore, R1, R2 and C3 act as the high-pass filter. The voltage divider between R1 and R2 helps adjust the voltage gain of the filter.
- At high frequencies, the impedance of C4 is very low, which allows much of the high frequency signal back into the op-amp, which causes the op-amp to block the signal. Therefore, C4 and R5 act as the low-pass filter.

Designing the filter

Designers may refer to the transfer function in [Appendix 4.6](#) to calculate the appropriate resistor and capacitor values to design the filter.

A unity-gain design was chosen for the MFB filter as easy gain adjustment was deemed necessary for this project. As it was difficult to adjust the gain of the MFB filter with a simple potentiometer, an inverting amplifier was added after the MFB filter to achieve a simple way to adjust voltage gain. A gain range of approximately 1-20 was deemed suitable for satisfying the design objective.

Designers may choose to use any capacitor value of their choice. A capacitance value of between 10nF – 100 nF was shown to yield suitable resistance values. Both capacitors can be of the same capacitance. Designers may choose to add capacitors at $\pm VCC$ to reduce noise.

First implementation of filter

The first MFB filter was designed with a maximum bandwidth of 1000 Hz. A capacitance value of 22 nF was chosen due to the ease of procurement from the fabrication laboratory and suitable corresponding capacitance values. The following resistor values were calculated, and the circuit was implemented in Multisim. 10 μ F capacitors were added to the $\pm VCC$ terminals to reduce power supply noise.

$$R_1 = 7234 \, \Omega \quad R_2 = 212 \, \Omega \quad R_5 = 14468 \, \Omega$$

The schematic and bode plot for this filter design may be referred to in [Appendix 4.6](#). A Bode plot was generated in Multisim to verify the filter's accuracy and bandwidth. While the filter design looked good on paper, the physical implementation, with the microphone and ADC connected showed that the actual bandwidth was around 1600 Hz, which was beyond the maximum threshold needed for the design. Therefore, the need for a redesign was deemed necessary.

Final implementation of filter

The final implementation of the MFB filter was designed with a maximum bandwidth of 500 Hz to fix the problem encountered with the first filter design. The 22 nF capacitors were replaced with 33 nF capacitors as they were larger in size and physically easier to work with on a breadboard. The 10 μ F capacitors were removed from this implementation as a space-saving measure on the physical breadboard. The following resistor values were calculated and implemented in Multisim.

$$R_1 = 9645 \, \Omega \quad R_2 = 70 \, \Omega \quad R_5 = 19291 \, \Omega$$

The final filter schematic was designed and tested in Multisim. A bode plot was generated to in Multisim to verify its accuracy. The filter was then built onto a physical breadboard and tested using a bode plotter on a bench-top oscilloscope.

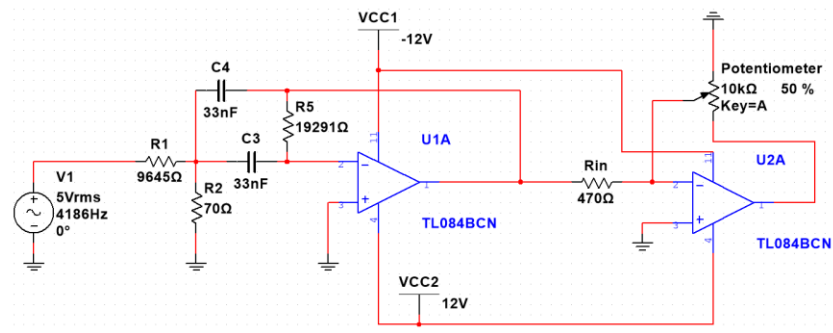


Figure 4.6.2: Final Filter Circuit Schematic

Multiple resistors connected in series to match the resistor values from the Multisim schematic. Designers can opt to use potentiometers in place of resistors too.

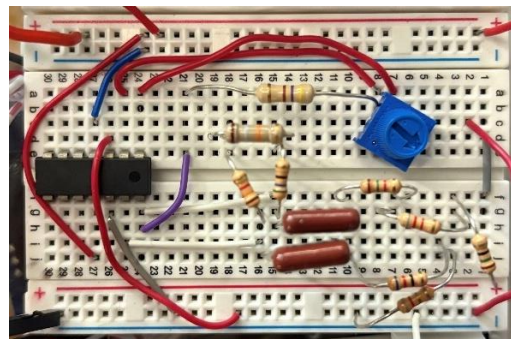


Figure 4.6.3: Final Filter Design on Breadboard

The bode plots for the Multisim schematic and physical breadboard were observed to be very similar to each other. The filter was tested with the ADC and Microphone and the bandwidth was observed to be around 700 Hz.

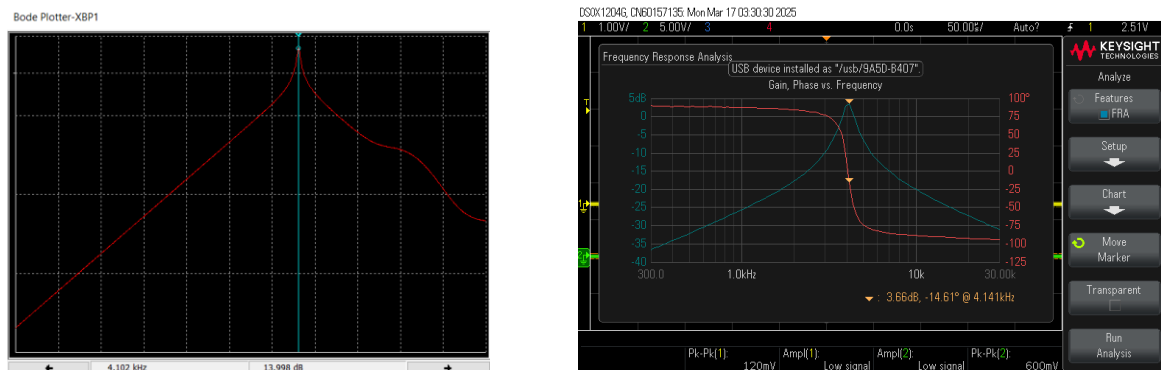


Figure 4.6.4 a) and b) Bode plots for Multisim Simulation and Physical Breadboard Implementation

4.7 – Half Wave Rectifier

Objective

The objective of the half-wave rectifier is to convert the AC signal from the filter's output into a DC signal that can be utilized with an ADC.

Why is rectification needed?

The filter outputs a sine wave, meaning it oscillates between positive and negative voltages. If a raw AC signal were directly input into an ADC that utilizes comparators, the comparators would rapidly switch between a logic high and logic low at the frequency detected. This creates square waves that resemble PWM (Pulse Width Modulation) signals rather than a steady DC signal.

When a GPIO pin is set to input it will not measure the signal correctly as the square wave cannot be used to measure the mic's signal strength, it will be at a floating state. To prevent this, a DC signal will be provided that can consistently represent the voltage output of the filter, which then can be utilized with an ADC to correctly detect the signal strength of the mic using voltage thresholds.

Full Wave versus Half Wave Rectifier

To convert an AC voltage into a DC signal for the ADC, there are two rectification methods that are suitable for satisfying the objectives: Full-Wave rectifier and Half-Wave rectifier.

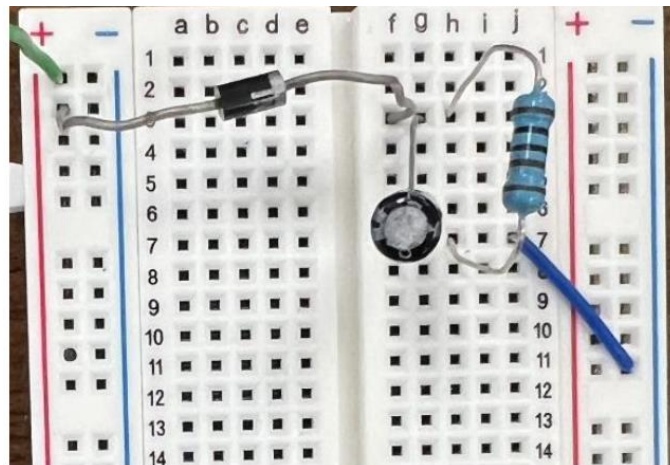
A Half-Wave Rectifier uses a single diode that blocks the negative swing of an input AC voltage. When the AC input is in its positive swing the diode allows current to pass, but the diode blocks conduction when the input is in its negative swing. What results is a waveform that only includes the positive half of the AC input and follows it with a capacitor that can smooth the waveform and output a steady DC voltage.

A Full-Wave Rectifier utilizes four diodes in a bridge configuration which inverts the negative swing of the AC input, and both halves of the AC input are output. The result is a waveform with continuous positive swing. Following it with a capacitor will smooth the DC output and offer a higher average DC voltage.

Half Wave Rectifier Implementation

The Full Wave Rectifier being the better choice as the amplitude of the analog input can be kept. The implementation is more complex as the bridge configuration could not rectify the signal. Implementing the half-wave rectifier produced the expected output and decided to utilize the half-wave rectifier. The rectifier circuit consists of a diode in series with a load resistor of $1\text{ k}\Omega$, followed by a 100 nF capacitor in parallel with the resistor. The capacitor is used to reduce the ripple and smooth out the waveform to produce a steady DC voltage. The reference schematic for a half wave rectifier is shown in [Appendix 4.7.1](#).

One issue faced with the half wave rectifier was the voltage loss at the output due to the diode's forward voltage drop of 0.7 and the loss of the negative swing of the AC input reduces the amplitude. To combat these issues, an amplifier stage was introduced before the half wave rectifier to change the gain of the filter's output. This allows a boost in the amplitude of the AC voltage before the rectification stage to a level sufficient to be input into the ADC after the voltage loss of the half wave rectifier.



4.7.2: Hardware Implementation of Half Wave Rectifier

4.8 – Analog to Digital Converter (ADC)

Objective

The primary objective of the Analog-to-Digital Converter (ADC) is to convert the analog signals from the microphone and filter stages into digital logic levels that can be interpreted by the Raspberry Pi 4.

In the context of the Sound Tracking Autonomous Rover (S.T.A.R.), the ADC will act as an interface between the analog sound output and the digital state of the Raspberry Pi. After the sound is captured by the microphone and passes through a filter and signal processing stage, the ADC should convert the resulting signal into a 2-bit digital signal.

Why is an ADC Necessary?

The Raspberry Pi 4 has no built in Analog to Digital Converter, it only has digital GPIOs, meaning each input pin can only read a logic 0 or a logic 1. The output of our filter is still an analog output so it cannot be directly processed by our Raspberry Pi 4, which can only read digital inputs, this is where our ADC comes into place.

Choosing an ADC

When choosing an ADC there are two key factors that must be considered:

Resolution and **sampling rate**.

- **Resolution** determines the smallest increment, and analog signal is divided into discrete digital levels. Higher resolution allows for more representation of small changes in signal.
- **Sampling Rate** refers to the samples per second the ADC can read from the analog signal. Referring to the Nyquist theorem, a sampling rate must be at least twice the highest frequency captured to avoid aliasing, which occurs when a signal is sampled at a lower rate leading to distortions in frequency readings.

In detecting a specific frequency of C8 (4186 Hz), it's important to choose an ADC that could sample fast enough to respond to the signal in real time and have enough bit levels to distinguish between different levels of voltage. However, this needs to be balanced against hardware complexity and the GPIO limitations on the RP4. This constraint narrowed our search to two ADC: Successive Approximation Register (SAR) and the Flash ADC.

A SAR ADC works by utilizing a binary Search with a digital to analog converter (DAC), and a comparator loop. It starts by setting the most significant bit to 1 and all other bits to 0, which then the DAC generates a voltage for that digital code. Then the comparator checks whether the input voltage is higher or lower than the DAC output voltage.

- If the input voltage is **greater** than the DAC output, the comparator outputs a logic high, and the bit remains a 1.
- If the input voltage is **less** than the DAC output, the comparator outputs a logic low, cleared back to 0.

The ADC continues down each bit till it reaches the least significant bit (LSB) and yields a high-resolution bit output.

A Flash ADC works by utilizing comparators, resistor ladder, and a priority encoder. The ADC works by dividing a voltage reference using a resistor ladder which connects to various comparators acting as thresholds. The comparators then check whether the input voltage is greater than or lower than the threshold set. The comparator will output 1 if the input voltage is greater than the threshold set and will output 0 if lower than the threshold. This isn't yet turned into a digital state but rather thermometer code depending on how many bits needed is input into a priority encoder which then does all the conversion necessary to turn into a digital binary output.

Evaluating the advantages and limitations of both the SAR and Flash ADC, the team decided to implement a Flash ADC. While the SAR offers a high resolution up to 16 bits such resolution was not deemed necessary for the application's requirement. The ADC requirement is to simply distinguish between weak and strong signals. Additionally, the SAR requires multiple clock cycles per conversion, as well as a dedicated control logic, which introduces additional complexity in hardware and software.

In comparison to Flash ADC, it provides a more straightforward implementation. Since the input signal is simultaneously evaluated by all comparators it offers the advantage of real-time voltage comparison, which eliminates the risk of aliasing. The main drawback of the Flash ADC is the limited resolution, since the number of output bits correlates to the number of comparators required, for example, an 8-bit Flash ADC would require 255 comparators. Since the resolution required is only a 2-bit ADC this would only require 3 comparators so the main drawback of the Flash ADC would not affect the requirements needed.

Implementing Flash ADC

Once an ADC is selected, the team begins the implementation process by identifying the required components needed to build a Flash ADC. Referencing a simple 2-bit Flash ADC schematic as a starting point to understand the required configuration of the necessary components.

The initial Flash ADC design was based on a publicly available schematic, which is included in [Appendix section 4.8](#). This reference provides a foundation for understanding the component layout and behavior of a 2-bit Flash ADC.

Using the reference schematic the components require are as follows:

- Voltage reference
- 3 op-amps
- Resistor Ladder
- Priority Encoder

Using the 12V power supply as the voltage reference, a series of resistors – One 4.3 k Ω followed by 1 k Ω resistors- were used to step down the 12 V reference voltage into various voltage thresholds that will be input into each comparator. A LM339 is used as a quad comparator to signify the 3 op-amps. The priority encoder will be integrated through the RP4 using no hardware whatsoever.

Using the resistor values provided, the voltage threshold for each comparator can be calculated using a voltage divider formula referred to in the [Appendix section 4.8](#).

The following table is produced using the voltage divider formula to signify the threshold voltage at each inverting pin meaning that the voltage must be greater than the set voltage for the comparator to output a logic high (1).

Analog Voltage	Comparator 3	Comparator 2	Comparator 1
0V-1.67V	0	0	0
1.67V – 3.29V	0	0	1
3.29V – 4.93V	0	1	1
> 4.93V	1	1	1

Table 4.8.2: Analog Voltage versus Comparators Logic Table

Understanding the certain voltage requirements needed to output a high from the comparator, each resistor going down the ladder is connected to the inverting pin of our comparator and the analog signal goes into each non-inverting pin of the comparator.

To build the circuit the quad op amp's pinout is needed to determine what pins need to connect. Analyzing the datasheet to obtain the pinout the pins 4, 6, and 8 are inverting pins and our resistor ladder will connect to each pin. To build the circuit a 12 V supplied by the power supply will connect in series to a 4.3 k Ω resistor followed by three 1 k Ω resistors in series creating a resistor ladder, at each node will then connect to pin 4, 6, and 8 respectively. This sets the threshold voltages for each comparator and pins 5, 7, and 9 are non-inverting pins which will all receive the rectified analog input from the filter. The quad amp will need to be powered on with pin 3 being the positive supply and pin 12 is grounded. Pin 3 will receive 12 V from the power supply and pins 1, 2, and 14 will be the output of the comparator. Each output from the comparator can be measured using an oscilloscope measuring the DC voltage of the comparator as this will act as a GPIO pin reading the output.

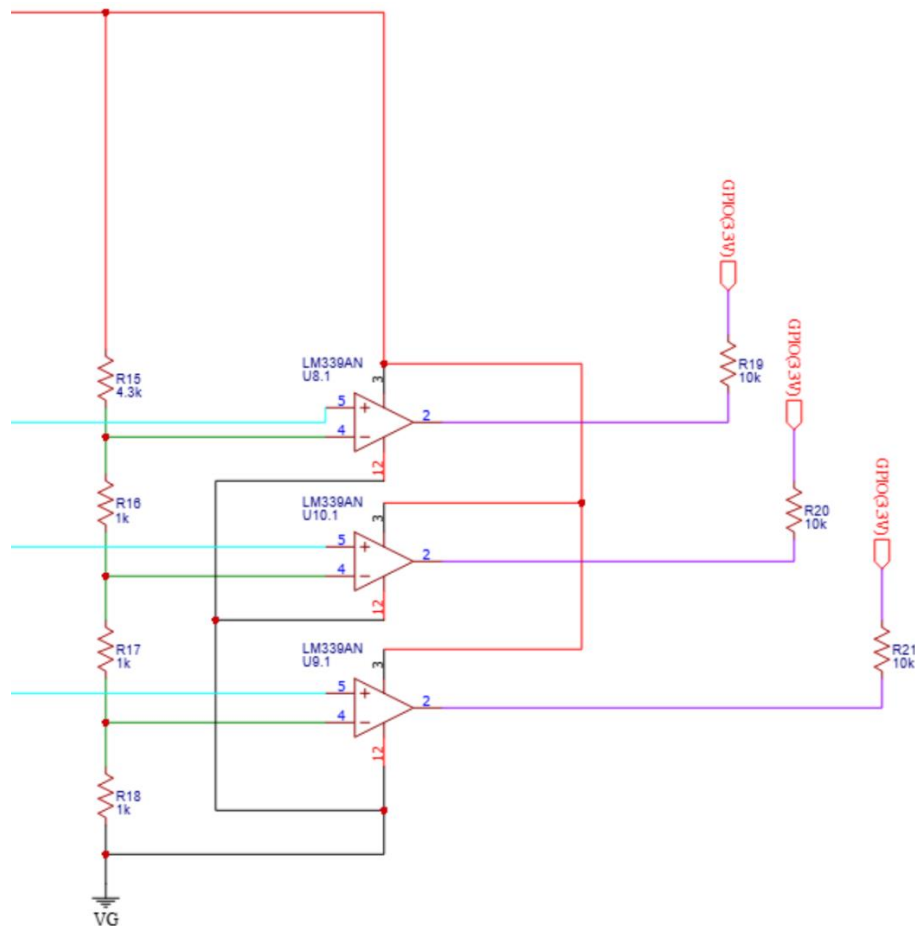


Figure 4.8.3: Final Flash ADC Schematic

When testing the ADC, a function generator is used to generate an AC signal which is then rectified using the half wave rectifier. The DC signal is inputted into each comparator at various amplitudes to obtain the desired output.

When testing the LM339N comparators were not producing the valid high output that should theoretically output. Analyzing the datasheet of the LM339N the behavior was linked back to an important feature of the LM339N, is it having open-collector outputs. This means that the output transistor inside the comparator can only pull the output pin to low or a 0 V state. When the transistor is off, instead of showing a logic high (1), the output pin is disconnected and in a floating state. To ensure that the output pin can be pulled to a high state a pull-up resistor is utilized. Each output pin for each comparator is connected to a 10 k Ω resistor that is then connected to a voltage of 5 V. This ensures that when the transistor is off, instead of a floating state it will pull the output to our 5 V supplied by the RP4.

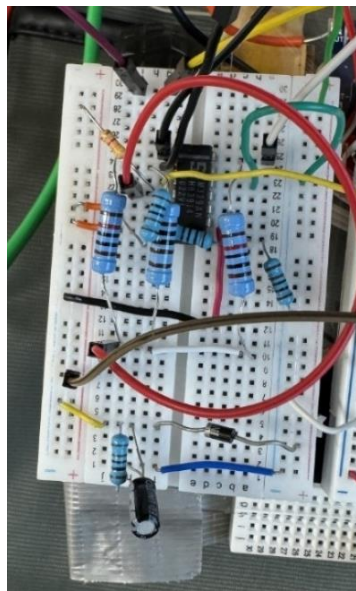


Figure 4.8.4: Final Flash ADC Hardware Implementation

Testing the ADC

Now that the hardware implementation is done, to test the ADC various rectified signals are sent representing the analog input of the ADC. Referring to [Table 4.8.2](#), the ADC was confirmed to work. At a voltage between 0 and 1.67 V none of the outputs were 5 V, until it crosses that threshold the first comparator would output a logic high or in this case 5 V. Going through each voltage threshold the comparators would turn on one by one and a voltage of over 4.93 V would have every comparator on which represents our highest binary state of 11. [Appendix Figure 4.9.5](#) showcases the output of the comparators at a voltage of 2 V which represents the yellow waveform. The green line representing the first comparator can be seen to be at 5 V, furthermore the blue line represents the second comparator, and the red line represents the last comparator. In the figure both blue and red channels are at a low state, moving on to figure

4.9.6 and 4.9.7 the output of the comparators turn on one by one when the voltage requirements are met.

To integrate the ADC to represent different levels of signal that our mic will receive, the mic, filter, and rectification are utilized. The rectified signal of the filter output will be inputted to each non-inverting pin. To represent different signal strength, the desired frequency is played 3-5ft away and the filter output is then rectified and inputted into our ADC. The idea of utilizing the ADC will be to output a binary output of 11 when the frequency is detected by the mic in a straight line while all other directions would output a lower binary output. To achieve the rectified signal must be greater than 4.93 V when the desired frequency is played 3-5 ft away only when the mic is directly straight to the frequency source. This is achieved with a few tests of setting up the frequency source at a 3-5 ft and having the mic facing straight to the source. The output of the rectified signal was not near 5 V and would need to be amplified by an amplifier to obtain the desired 5 V. After the amplification test the desired 5 V was achieved and inputting this into our ADC will result in a binary output of 11. Further testing would include having the frequency source to the right or to the left of the mic and observe the output voltage after amplification and indeed the drop off was significant enough to not output a binary output of 11 so the ADC fully achieves the necessary requirements by diving different signal strength that will be read from the RP4.

4.9 – Bump Switch

Objective

The objective of the bump switch is to act as a physical collision detection mechanism for the S.T.A.R. Positioned at the front of the vehicle, the bump switch detects direct contact with obstacles. When triggered, it signals the Raspberry Pi 4 to halt all motor movement, preventing potential damage and adding a layer of safety to the system's navigation behavior.

Understanding the Bump Switch

The bump switch is a normally open mechanical switch that closes the circuit when pressed as shown in [Figure 4.8.1](#). It acts as a digital input device.

- When the switch is not pressed, the circuit remains open, and the Raspberry Pi reads a LOW (0V) signal.
- When the switch is pressed, the circuit closes and pulls the GPIO pin to HIGH (3.3V), which the Pi interprets as a digital HIGH signal.

This binary behavior makes it ideal for quick, reliable collision detection with minimal software overhead.



Figure 4.9.1: Bump Switch

Wiring and GPIO Configuration

The bump switch used in this design features three terminals: Common (COM), Normally Open (NO), and Normally Closed (NC). For this application, only the COM and NO terminals were used, while the NC terminal was left unconnected. This is because the intended behavior is for the switch to remain open (OFF) during normal operation and only close (ON) when physical contact occurs—triggering a HIGH signal on the Raspberry Pi.

The bump switch is connected to [GPIO Pin 20](#) (Physical Pin 38) on the Raspberry Pi as shown on in. The COM terminal is connected to the 3.3V rail, and the NO terminal is connected to GPIO 20. A pull-down resistor configuration is used in software to ensure the default state reads LOW when the switch is unpressed. When the switch is pressed, it completes the circuit, allowing 3.3V to be read as HIGH by the GPIO pin.

Mechanical Integration

To ensure reliable contact detection, the bump switch was mounted to the front center of the chassis, as seen in figure 4.10.2, using a custom 3D-printed bracket, as shown in appendix 4.10.1. The mount was designed to hold the switch securely in place while allowing unobstructed movement of the actuator when pressed. This placement ensures the switch is triggered upon head-on collisions or frontal contact with the 4186 Hz frequency sound.

To increase the effective area of contact and ensure consistent triggering when approaching a surface, the team attached a plastic spoon and fork to the actuator of the bump switch. This simple but effective modification extended the reach of the switch and helped ensure activation even if only part of the car touched an object. The spoon and fork acted as lightweight, flexible bumpers that would flex or bend slightly upon contact, reliably pressing the switch and triggering the stop condition.

Behavior and System Response

When the bump switch is pressed, the Raspberry Pi 4 immediately halts all motor movement by disabling the H-bridge input and enable pins. Simultaneously, all four onboard LEDs blink three times in unison to provide a clear visual indicator that a collision has been detected. This blinking sequence serves as immediate feedback to the user, confirming that the bump switch was successfully triggered and that the system responded accordingly. The behavior was implemented using Python, with the GPIO library used to toggle the LED states with short delays between flashes. Together, the motor shutdown and LED alert create an effective collision response system, adding both functionality and user-awareness to the STAR platform's real-time operation.

Section 5 – Printed Circuit Board (PCB) Design

Almost every consumer electronic product is implemented on a Printed Circuit Board (PCB). Breadboard wiring is quick to make and perfect for testing new designs physically, but they should only be used for temporary designs. A more permanent option is a circuit on a perfboard, but perfboards should also only be used for prototyping as there is always exposed solder and components are generally wired together crudely. PCBs are designed in software and professionally manufactured to exact specifications, making them perfect for both mass production and final designs.

5.1 – CAD Software Overview

Objective

The objective of this section is to explain the team's choice of CAD software and why. Multiple different CAD programs were suggested in the deliverable documentation, therefore the team needed to research each program to decide which to use. For the project's needs, the CAD program must have specific features to be effective:

1. Schematic design and node declaration
2. Symbol design for custom parts
3. Functionality for assigning models to parts on the schematic
4. Physical board placement designer with 3D modelling
5. Trace and via mapping in PCB layers
6. Design Rule Checker (DRC) for manufacturers

Choosing the Desired CAD Software

No one on the team had any experience with CAD programs before, therefore the simplest and easiest to learn program with all required features would be optimal. The first program that was suggested to the team was Fusion360 for its power, versatility, and availability to UGA students. The team followed multiple tutorials provided by Fusion and found that Fusion360 has a steep learning curve for beginners to CAD software. It may be powerful and versatile, but the team only needs it for one specific purpose and would not be using it to its full potential. Therefore, a simpler option with less extra features would be more efficient to learn and use. The team looked towards KiCAD as a potential option.

Appendix Figure 5.1.1. The team found that KiCAD was quick to learn and had the required tools, KiCAD became the CAD software of choice.

5.2 – CAD Schematic Design

Objective

Now that the team has chosen a CAD program and is familiar with how to use KiCAD, the final schematics of every circuit in the project need to be implemented with KiCAD's symbols and connections. Every component will have a value, assignment number, connection label, footprint. The footprints will be discussed in later sections.

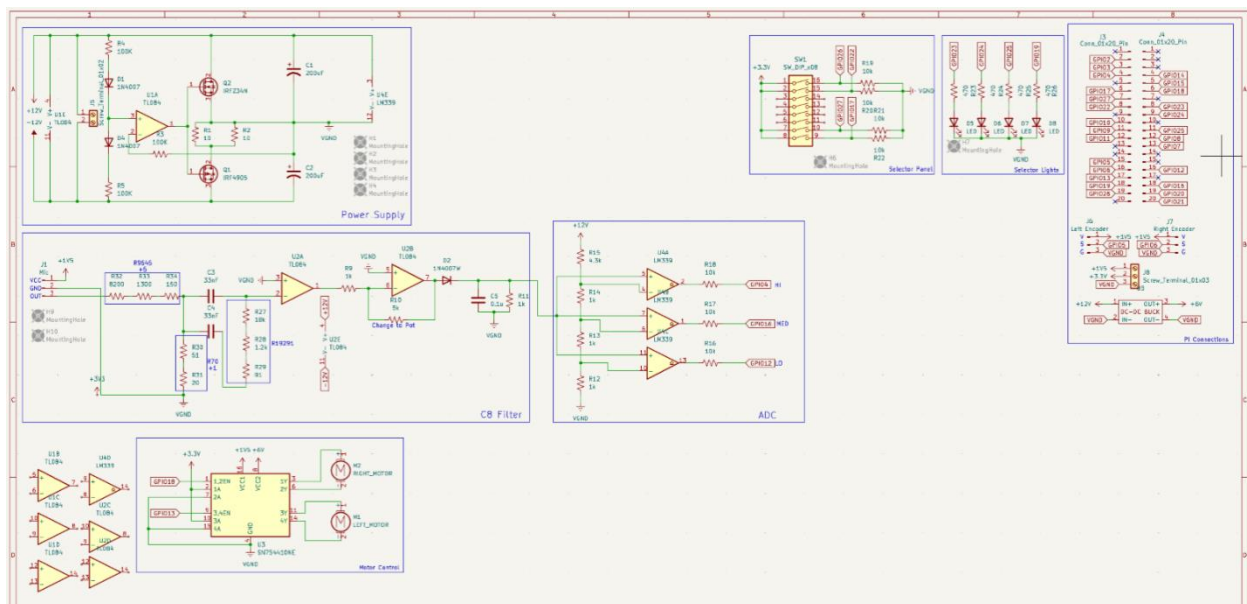


Figure 5.2.1: Full schematic design implemented in KiCAD

Schematic Design Aspects

KiCAD has many important steps that must be taken to ensure proper functionality and formatting for all parts. The first important aspect of KiCAD is nets. A net in KiCAD is an address to each specific node in the circuit diagram. Every part that is connected to each other physically must be in the same net. Nets in the schematic are represented by the green lines connecting each part. If every single net was connected with visual green lines throughout the entire schematic however, the schematic would be difficult to read. Labels are the solution to different sections of the schematic needing connected nets.

A label makes one net able to be accessed throughout the entire schematic. Having labels throughout the schematic allows each individual circuit to be separated visually while still being functionally connected. A third critical aspect of the schematic is that realistic components have specific values. In simulations and drawn schematics, values of components like resistors can be set to anything, but real components have discrete values. To compensate, multiple components must be used to achieve the desired values. As seen in [Appendix figure 5.2.2](#), multiple resistors of values that can be realized were connected in series. For example: A desired resistance of $9650\ \Omega$ was required, therefore common resistances with resistances of $8200\ \Omega$, $1300\ \Omega$, and $150\ \Omega$ can be connected in series.

5.3 – Component Placements

Objective

With the schematic fully designed, the next step is to get the realized PCB designed and ready to be sent off for manufacturing. Multiple actions are required to get the design ready for manufacturing: each component needs to be assigned its real-life counterpart, then the physical board dimensions and shape must be created.

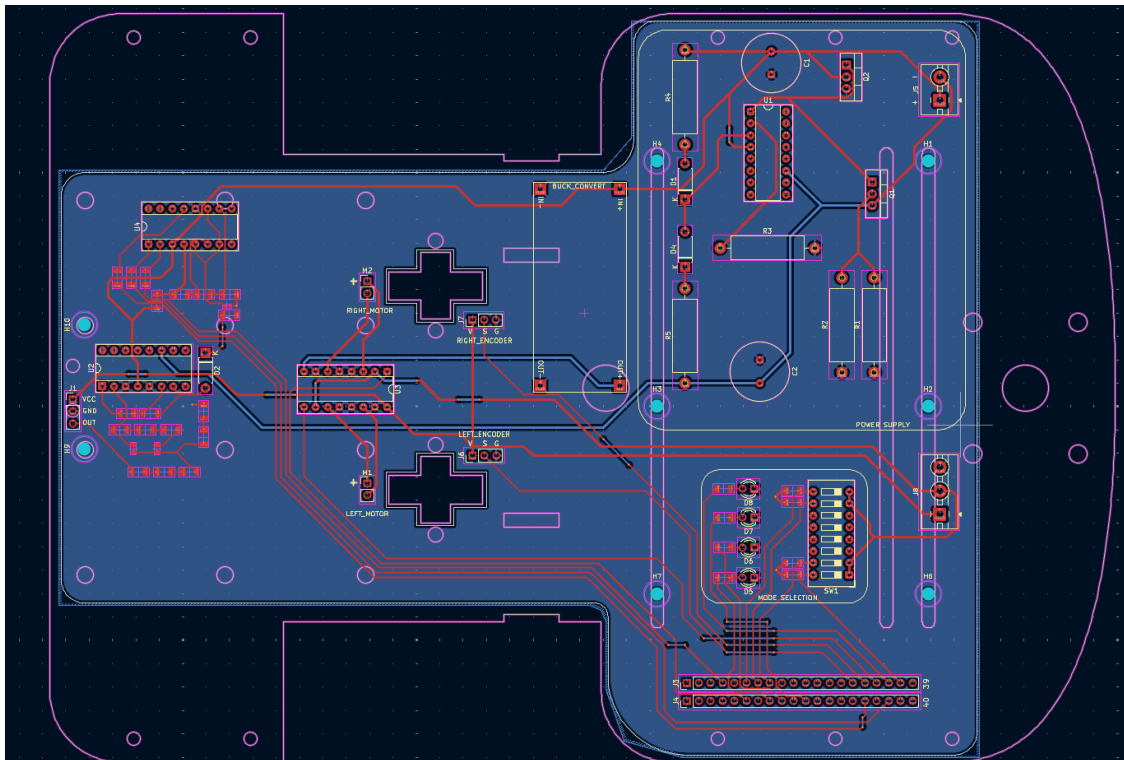


Figure 5.3.1: Full PCB design implemented in KiCAD

Assigning Component Footprints

Before the PCB can be designed, each symbol of the schematic must be assigned a corresponding footprint. A footprint is the physical model and dimensions a component will have in the physical realization. A full list of the components and corresponding footprints can be found in [Appendix Table 5.3.2](#).

KiCAD has default libraries of most common components, therefore the team could pick proper components from a supplier or directly from the manufacturer and match the footprint to the component. A more detailed example of a component, the 8 pin DIP switch, can be found in [Appendix Figure 5.3.3](#). The footprint including the proper pins for the component as well as dimensions appear on the left side of the figure, and the corresponding 3D model can be seen on the right side. It is crucial to use the proper footprint in a PCB design with the correct pin diagrams, as each IC chip could have different pins for different functions.

Placing and Routing Components

The next step in PCB preparation is setting up the PCB editor with design rules and constraints according to a manufacturer. JLCPCB was recommended to the team as a PCB manufacturer, therefore the design constraints were added into KiCAD from JLCPCB's website, as seen in [Appendix Table 5.3.4](#). Once the board design was set up, a drawing of the S.T.A.R.'s acrylic platform was inserted into the PCB sheet so the team could design the board to fit on top of the platform.

There are multiple approaches possible for creating the PCB for the S.T.A.R. The team could prioritize each section of the schematic design and have multiple separate smaller PCBs connected with wires, or the design could be centralized onto one larger board. The team ultimately decided to put the whole schematic onto one PCB to minimize wire connections and failure points. An advantage to one larger PCB as well is that the bottom layer of the PCB can become a ground plane which minimizes possible noise and ground loops. To make the PCB easy to follow however, each main section of the schematic was placed in separate locations on the larger board. An example of which is the isolated power supply circuit which can be found on the top right of the PCB design.

To make sure that every part is present and correct, it is good practice to move all parts into a rough position before routing traces. KiCAD has a feature called "rats' nest" which displays all required connections without routing any traces, which allowed the team to efficiently put components with optimal connections. Once every component is in a generally correct position, final adjustments can be made and traces can be routed. There are two widths of traces on the PCB, a smaller width for signals and a larger width for power distribution.

The team decided to use a header so that the Raspberry Pi's GPIO cable can directly connect to the PCB. Any trace that would cross with another must have a via. A via connects the top layer of the PCB to the bottom layer, effectively allowing traces to bridge over one another to keep traces from crossing each other. To minimize noise, the via bridges are made as short as possible. Signal traces are placed parallel to one another, when possible, to maximize readability of the PCB.

5.4 – Preparation for Manufacturing

Objective

The final step before ordering a PCB from the manufacturer that the team chose, JLCPCB, is uploading the proper files to their website to pass their internal Design Rules Checker, create a Bill of Materials (BOM), and quote the team on cost and shipping of the PCB.

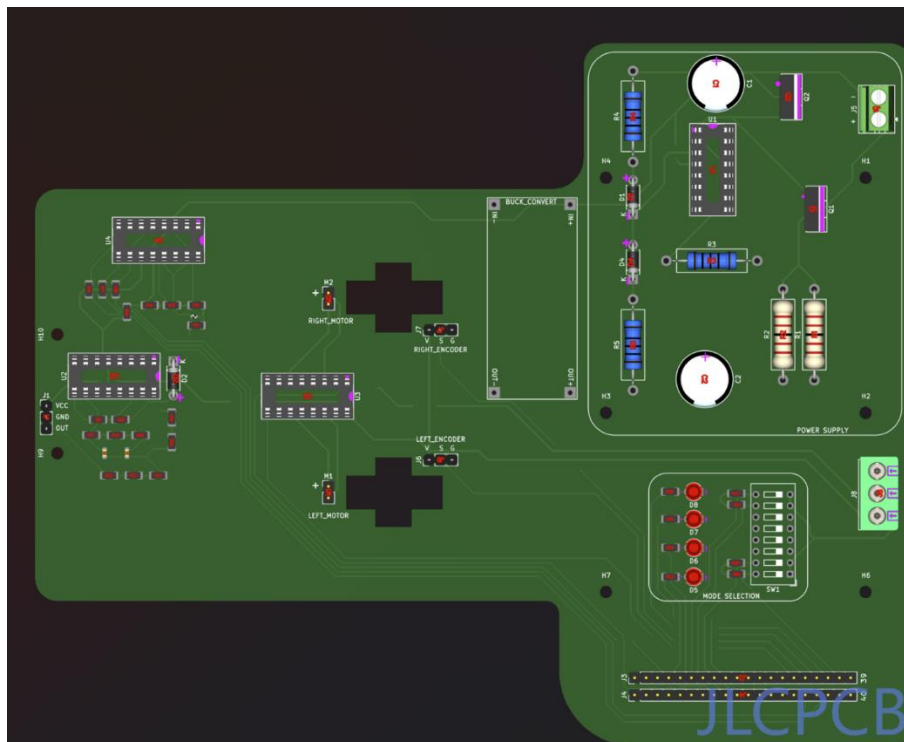


Figure 5.4.1 3D render of all parts placed onto the board

Uploading Files to JLCPCB

Earlier in section 5.3, the component footprints that were assigned were chosen with the manufacturing in mind. JLCPCB has a vast library of parts that can be selected for use with the PCB. The parts library allowed the team to select the component and specifications, then choose what parts were physically available. Each specific part has a unique library number that can be entered in the Bill of Materials (BOM) to allow quick access from the manufacturer. Each part also has a corresponding footprint model that links to KiCAD. Once the full schematic was finished and KiCAD's internal Design Rule Checker (DRC) gave no errors, the team generated Gerber files to upload to JLCPCB for its own DRC. Once the footprint board schematic was cleared, the physical 3D models had to be placed onto the board in another check to make sure orientation was also correct. **Figure 5.4.1** shows a screenshot of the 3D rendering on JLCPCB before the final quote.

Final Quote for Manufacturing and Parts

The final quote was instant and gave a price for each individual section of the PCB assembly. The price was notably higher than the team expected, mainly due to the minimum order requirement of 5 complete boards and the Extended components fee. The extended components fee of \$42.00 was stated as labor required to set up custom commands for the pick and place machines for any nonstandard part. Most components on the bill of materials were listed as such, causing the bulk of the expense to go towards assembly, which can be seen in **Appendix Table 5.4.2**. If the team were to purchase the PCB, the team would purchase the components and physical board unsoldered separately. The team would then solder each component individually to minimize the cost.

Section 6 – Ethics

As students, we recognize the responsibility to uphold the ethical standards outlined in the N.S.P.E. (National Society of Professional Engineers) Code of Ethics. Throughout the development of this project, our team made deliberate efforts to ensure originality, integrity, and transparency in both our hardware design and software development processes.

From the beginning, our circuitry was designed independently through experimentation, research, and iteration. The 3D printed bump switch attachment was designed from scratch. Similarly, all code for this project was written and tested by us to reflect our own understanding and engineering judgment.

Collaboration with peers was conducted ethically and professionally. We engaged in open, conceptual discussions with other teams, exchanging general ideas and exploring possible design strategies. However, we were careful not to share or replicate specific implementation details, schematics, or code. This ensured that each team-maintained ownership over its design choices, promoting an environment of mutual respect and academic honesty.

Where external resources were referenced—including datasheets, tutorials, and open-source schematics—we took care to verify, interpret, and adapt these sources to suit our project goals. Proper citations were included in our documentation to acknowledge the original creators and avoid misrepresentation or plagiarism.

We also utilized AI-powered tools such as ChatGPT during the development process. These tools were primarily used to explore technical concepts and help clarify the behavior of unfamiliar components or circuits. While some responses provided accurate and helpful insights, others were incorrect or misleading. However, encountering these inaccuracies prompted us to further investigate, verify information through datasheets and technical sources, and ultimately deepen our understanding of how each subsystem worked. Rather than relying on AI for answers, we used it to challenge our assumptions, guiding our research and helping us strengthen our problem-solving skills.

Looking beyond the academic scope, we also considered the ethical implications of S.T.A.R. as a potential real-world product. Should the system ever be developed for consumer use, additional ethical priorities would emerge notably in safety and privacy. STAR's current prototype features open circuitry and exposed components, which would require enclosure, insulation, and safety labeling to comply with consumer safety standards. Furthermore, if future iterations of STAR were to collect user or environmental data (e.g., audio input patterns or movement logs), transparency and informed consent would be essential.

In accordance with professional ethics, we would ensure that such data collection is disclosed, justified, and conducted with user privacy as a top priority. In all aspects of this project, we aimed to practice responsible engineering by balancing innovation, collaboration, and public welfare. Our goal was not only to build a functional autonomous system, but to do so with a strong ethical foundation that reflects the standards of the engineering profession.

Section 7 – Self Critique

This section focuses on team members offering critiques on their own personal performance for the duration of this project.

Aryan

I feel like I could have been more efficient with my time during the designing phases of the deliverables. For example, I made an overlooked and simple connection error during a design phase, and it took us hours to figure out what the problem was and fix it. Our organizational skills, especially at the beginning of the project, were inefficient and our work always seemed to be all over the place. But we began to categorize and organize our tasks and designs better as we reached checkpoint B, and looking over the lecture slides really helped us improve our approach.

Miguel

A major critique that has been emphasized from this project has been the my time management and component research. Being able manage my time and allowing for extra time is very helpful as many objectives take longer than expected, and making sure that I do not underestimate any part of any project. Especially working with a team that relies on my work output and knowledge.

Jay

Throughout the semester, my time management was sub-par. There were times for deliverables in which the rest of the team were waiting for my work to be done. If I had started earlier on individual parts of the work, then I believe that I could have kept up with the team better. At times, stress due to procrastination led me to be scatterbrained during crucial moments in which the team depended on me. My best work was around the middle of the semester. I believe that this is because I had time to get used to working with the team but was not fatigued like I was nearing the end of the semester.

Justin

While I'm satisfied and proud of the contribution I made to the project, I also recognize areas where I could've done better. Time management is something I still need to improve. I could have started the deliverables earlier and made better use often weekends to make consistent progress on the deliverables. Another area for growth is my circuit knowledge. While I learned a lot throughout the semester there is always room for improvement. Another area is my communication skill as I am often reserved, I want to work on explaining concepts more clearly to an experienced audience in presentations.

Derrick

One issue I've struggled with and was brought to my attention due to this class is time management. I've always been somewhat laid back in this area. Although I've made steady progress since starting college, the course showed me how much further I still need to go. It was also humbling. I was never one to excel in basic circuit analysis and electronics, yet I felt confident in my conceptual grasp and believed I was better suited to practical projects that applied my circuit-analysis skills. The class confirmed that hands-on work does play to my strengths, but it also revealed that I had overestimated my knowledge of circuits and hardware. It was an eye-opener, highlighting how vital coursework is to practical understanding, regardless of the grade I earn. Despite the stress, I appreciate this class because it solidified my passion for my major when I was uncertain. ECSE 2920 may not be perfect, but it definitely motivates me to improve.

Section 8 – Revision Plan

Objective

The objective of the revision plan is to allow for multiple sound source detection and optimize the performance of the S.T.A.R.

Motors and H-Bridge

In the current version of S.T.A.R., the motors are configured for forward-only motion using fixed logic levels on the H-bridge input pins. This simplified our initial design and allowed us to focus on encoder feedback and speed control. However, as the project progressed, we recognized the need for greater flexibility in how the rover moves—particularly in scenarios like CHKP A-, where the system must approach an initial sound source (4186 Hz), then reposition to detect and navigate toward a second source (1046 Hz).

To support this behavior, we plan to rewire the L293 H-bridge so that the motor input pins are connected to GPIOs rather than being hardwired. This change would allow the Raspberry Pi to control motor direction in real time, enabling both forward and reverse movement while retaining PWM-based speed control on the enable pins. Enabling reverse motion will give S.T.A.R. the ability to back away from previously detected sources and reorient itself for more complex audio tracking tasks. This revision marks a key step toward more dynamic navigation and is part of our plan to expand the system's capabilities in future development.

Analog to Digital Converter

Reflecting on the current ADC design, potential revisions have arrived that could improve the performance of the S.T.A.R. A second ADC could be implemented to allow multiple sound detection as it would allow the RP4 to detect the levels of a second sound source. Another key improvement would be to increase the resolution of the current 2-bit resolution. A tradeoff of increasing the resolution, however, is the increase in the number of comparators used. A higher resolution would allow finer distinctions in signal strength of the sound source.

PCB Design

Due to the significant expense that manufacturing the PCB would be because of the large and nonuniform shape, the team would change the PCB design approach to individual smaller boards. The smaller boards would introduce more possible noise and failure points due to extra wires, but the S.T.A.R. Rev 1 worked as expected using only breadboards. The smaller boards would be more uniform and would drastically lower the cost of manufacturing and assembly.

Section 9 – Appendix

Appendix 2 – Hardware Constraints

Components	Quantity
Dupont Wires (FF, FM, MM)	As many as required
Wire Jumpers	As many as required
Breadboards	As many as required
GPIO Breakout with ribbon cable	1
24V Power Supply	1
Jack Adapter	1
Adafruit Auto Gain Mic	1
Quad H-Bridge (SN754410 or L293)	1
8-Switch DIP Switch	1
Bump Switch	1
LM339 Quad Comparator	1
Optical Encoders (sensor)	2
Optical Encoder (wheel)	2
TL084 Quad Op Amp	1
IRF4905 (P Channel MOSFET)	1
IRFZ34 (N Channel MOSFET)	1
LED Assortment	4
Assortment of Capacitors	As many as required
Assortment of Resistors	As many as required
Assortment of Diodes	As many as required
Raspberry Pi 4 (4GB)	1
Heat sinks	2
USB MicroSD Card Reader	1
32GB MicroSD Card	1
PI On/Off Switch	1
Pi Power Supply	1
Back Wheels	2
Front Wheel	1
Chassis	1
DC Motors	2
motor fasteners	As many as required
Screws and Nuts	As many as required
Spacers	As many as required

Appendix Table 2.1.1: Components used for the design of the car

Appendix 3 – Software

Bit 3 (GO)	Bit 2	Bit 1	Bit 0	Input Cases	Mode
0	X	X	X	0	GO/NO GO
1	0	0	0	1	Single Source
1	0	0	1	2	Multiple Source
1	1	0	0	3	Straight Line
1	1	0	1	4	Figure Eight
1	1	1	0	5	Square
1	1	1	1	6	Debug

Appendix Figure 3.2.1: dip switch bit configurations

LED GPIO	Bit #
GPIO 23	Bit 3
GPIO 24	Bit 2
GPIO 25	Bit 1
GPIO 19	Bit 0

Appendix Figure 3.2.2: GPIO pins corresponding to each dip switch bit

Appendix 4.5 – Push/Pull Amplifier

Voltage Divider Derivation

General Voltage divider formula

$$V_{out} = V_{in} \cdot \frac{R_2}{R_1 + R_2}$$

Setting $R_1 = R_2$:

$$V_{out} = V_{in} \cdot \frac{R}{R + R} = \frac{1}{2} V_{in}$$

Using Given values:

$$V_{mid} = 24V \cdot \frac{1k\Omega}{1k\Omega + 1k\Omega} = 24V \cdot \frac{1}{2} = 12V$$

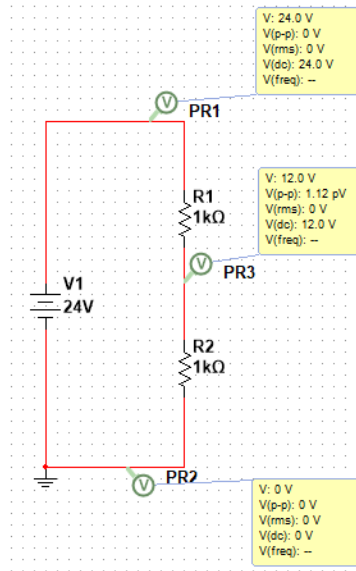


Figure 4.5.1: Power Supply Voltage Divider

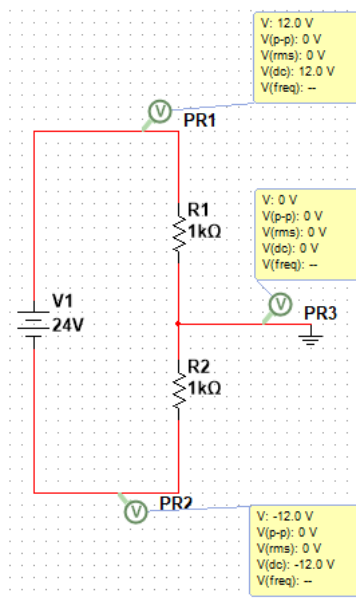


Figure 4.5.2: Midpoint Ground reference

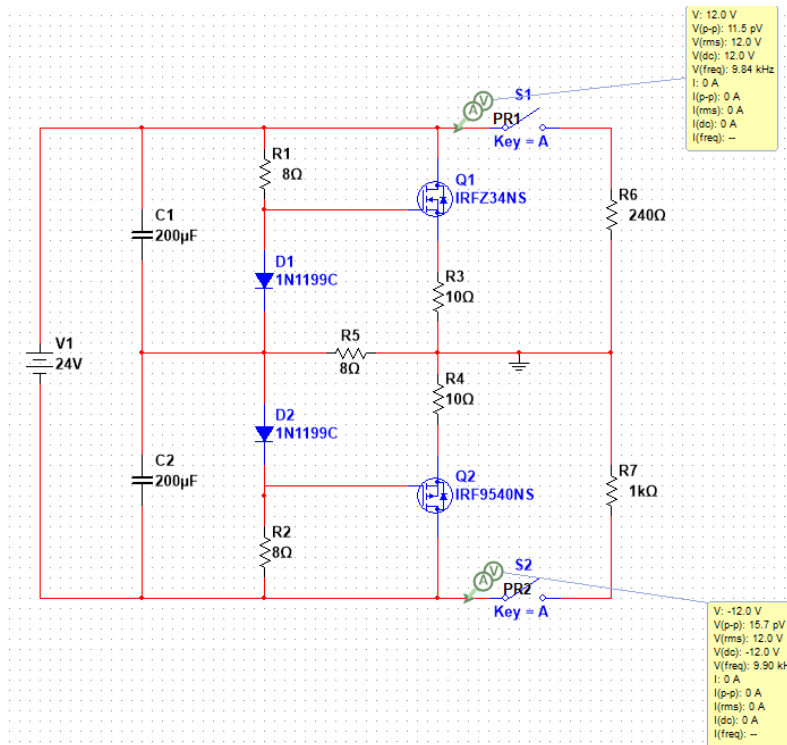


Figure 4.5.3: Initial Circuit Iteration

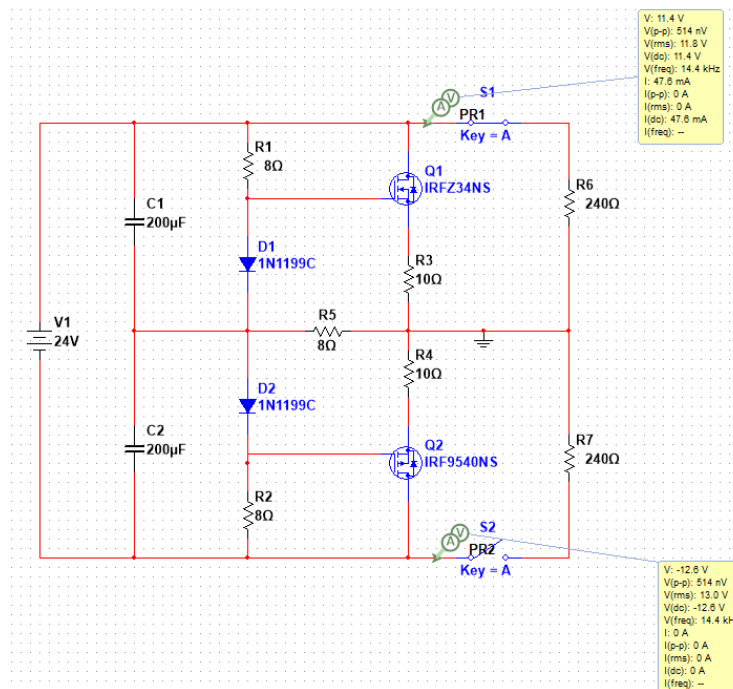


Figure 4.5.4: Initial iteration: Positive Load Connection

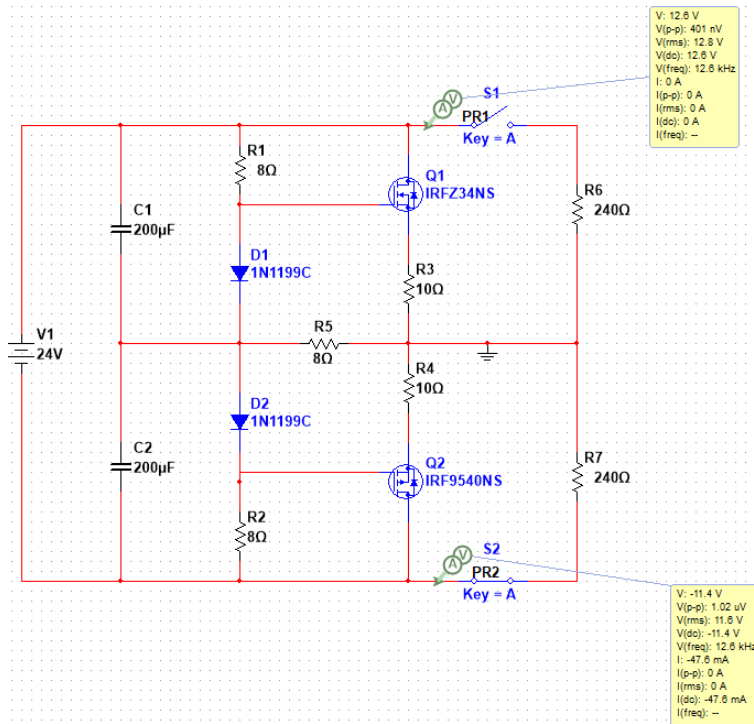


Figure 4.5.5: Initial iteration: Negative Load Connection

LOAD RESISTANCE(Ω)	Positive Power Rail (V)	Negative Power Rail (V)
5600	12	-12
4700	12	-12
2000	11.9	-11.9
1000	11.9	-11.9
560	11.7	-11.7
470	11.7	-11.7
330	11.6	-11.6
240	11.4	-11.4
150	11.1	-11.1
100	10.7	-10.7
47	9.56	-9.56

Table 4.5.6: Initial Circuit Iteration: Load Test

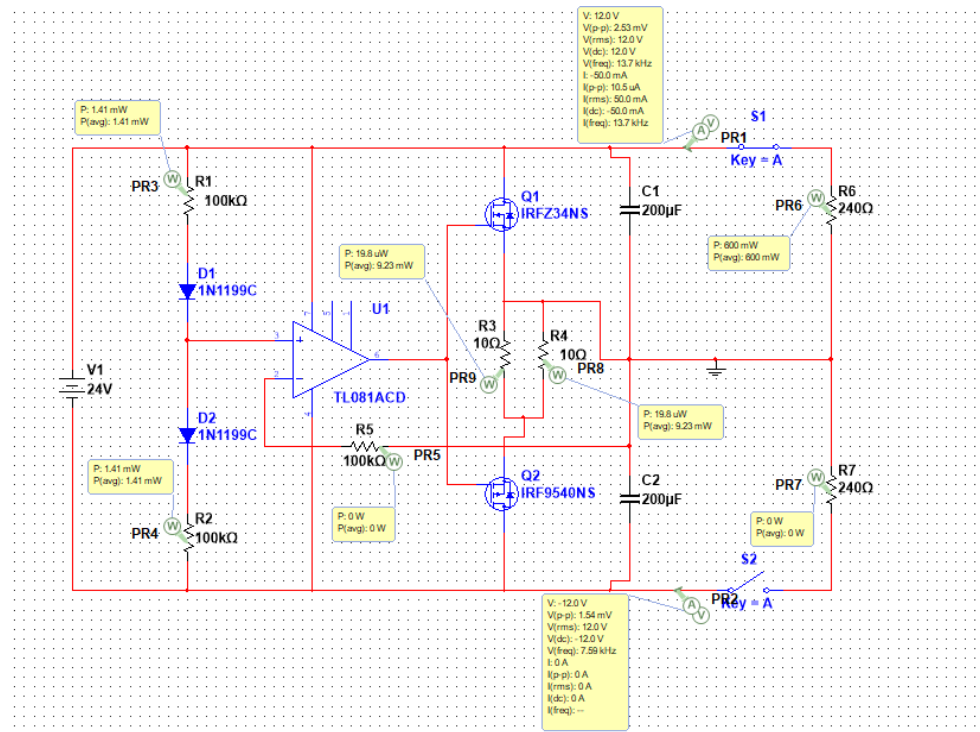


Figure 4.5.8: Final Circuit Iteration: Positive load Baseline test

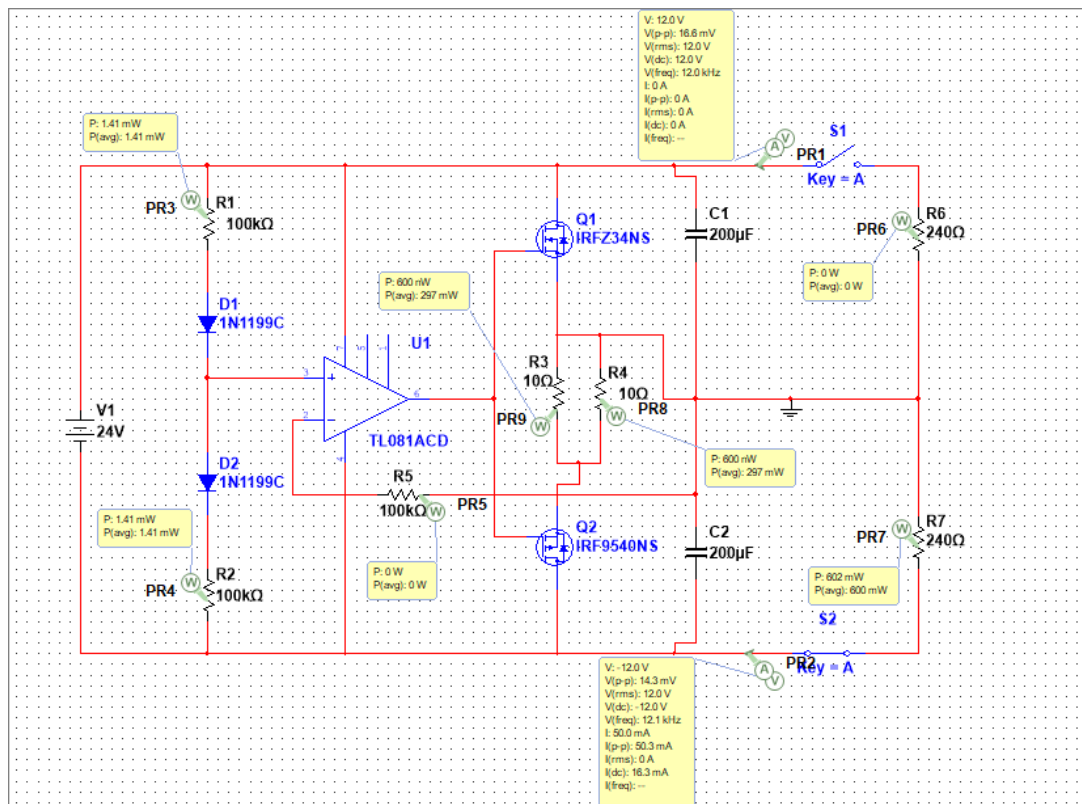


Figure 4.5.9: Final Circuit Iteration: Negative load Baseline test

LOAD RESISTANCE(Ω)	Positive Power Rail (V)	Negative Power Rail (V)
5600	12	-12
4700	12	-12
2000	12	-12
1000	12	-12
560	12	-12
470	12	-12
330	12	-12
240	12	-12
150	12	-12
100	12	-12
47	12	-12

Table 4.5.10: Final Circuit Iteration: Load Test

LOAD RESISTANCE(Ω)	VOLTAGE(V)	CURRENT(mA)
5600	12.15	2.1775
4700	12.1516	2.58
2000	12.15	6.07
1000	12.15	12.09
560	12.148	21.692
470	12.147	25.777
330	12.144	36.608
240	12.142	50.25
150	12.137	80.21
100	12.129	122.86
47	12.106	258.86

Table 4.5.12 : Physical Circuit Implementation: Positive Load Test

LOAD RESISTANCE(Ω)	VOLTAGE(V)	CURRENT(mA)
5600	-12.1712	-2.1808
4700	-12.17	-2.58
2000	-12.17	-6.088
1000	-12.169	-12.101
560	-12.167	-21.698
470	-12.166	-25.78
330	-12.164	-36.67
240	-12.16	-50.37
150	-12.153	-80.28
100	-12.144	-122.88
47	-12.111	-258.504

Table 4.5.13: Physical Circuit Implementation: Positive Load Test

Appendix 4.6 – Analog Filter Design

The transfer function of a 2nd order Multiple Feedback Bandpass filter is as follows.

$$H(s) = \frac{V_o}{V_{in}} = \frac{-s \frac{1}{R_1 C_4}}{s^2 + s \frac{C_3 + C_4}{C_3 C_4 R_5} + \frac{1}{R_5 C_3 C_4} \left(\frac{1}{R_1} + \frac{1}{R_2} \right)} \quad (1)$$

Comparing this equation to the standard form of a 2nd order Bandpass filter,

$$H(s) = \frac{K \omega_{BW} s}{s^2 + \omega_{BW} s + \omega_{center}^2}$$

We can derive the following equations.

$$K \omega_{BW} = \frac{1}{R_1 C_4} \quad \omega_{BW} = \frac{C_3 + C_4}{C_3 C_4 R_5} \quad \omega_{center} = \sqrt{\frac{1}{R_5 C_3 C_4} \left(\frac{1}{R_1} + \frac{1}{R_2} \right)}$$

Where,

K = Voltage Gain

ω_{BW} = Filter Bandwidth

ω_{center} = Center Frequency

The Quality Factor is defined as the ratio of the center frequency and the filter bandwidth.

$$Q = \frac{\omega_{center}}{\omega_{BW}}$$

The capacitors can be set to an equal capacitance to simplify the resistor calculation process.

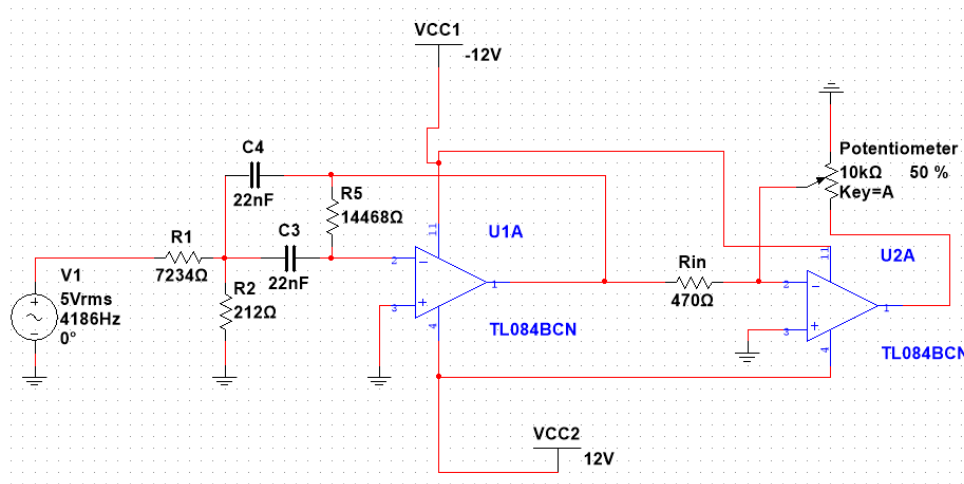
$$K \omega_{BW} = \frac{1}{R_1 C} \quad \omega_{BW} = \frac{2}{C R_5} \quad \omega_{center} = \frac{1}{C} \sqrt{\frac{1}{R_5} \left(\frac{1}{R_1} + \frac{1}{R_2} \right)}$$

Setting the Voltage Gain (K) to 1 for a unity gain setup, we can derive the following equations from the preceding equations.

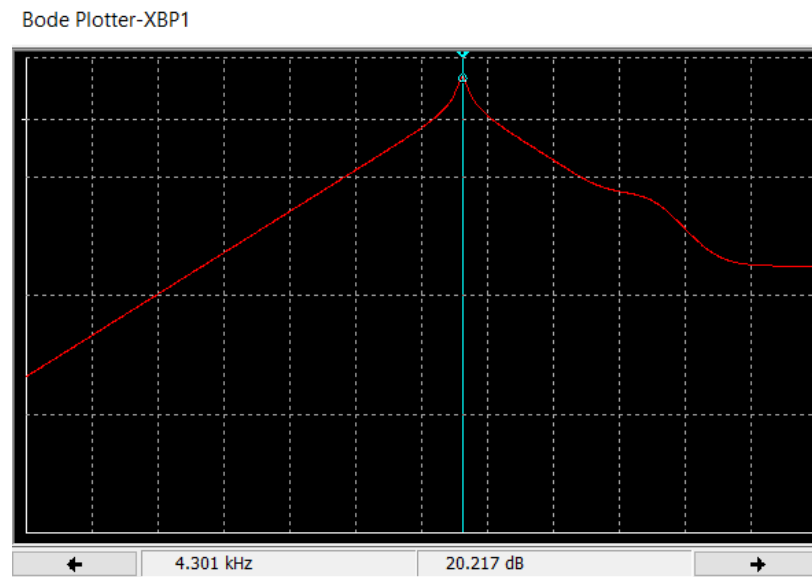
$$R_1 = \frac{R_5}{2} \quad R_2 = \frac{R_5}{4Q^2} \quad R_5 = \frac{2Q}{\omega_{center} C}$$

Designers can calculate the Quality Factor (Q), choose a capacitance value and calculate the resistor values to design their filter.

The schematic and bode plots for the first iteration of the filter design are as follows.

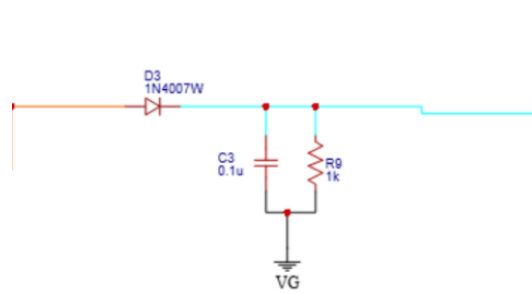


Appendix Figure 4.6.1: First Filter Design Schematic



Appendix Figure 4.6.2: First Filter Multisim Bode Plot

Appendix 4.7 – Half Wave Rectifier



Appendix Figure 4.7.1: schematic of half-wave rectifier

Appendix 4.8 – Analog to Digital Converter (ADC)

The voltage threshold of each comparator is calculated with the following voltage divider formula.

$$V_{out} = V_{in} \times \left(\frac{R_{below}}{R_{Total}} \right)$$

V_{out} - output voltage

V_{in} - The input voltage of 12V provided by the +12 V power supply

R_{below} - The total resistance under the node

R_{Total} - The total resistance of the resistor ladder this is known as $7.3k\Omega$

To calculate the threshold of the first comparator the resistance is calculated and input into the formula.

$$R_{below} = (1k\Omega + 1k\Omega + 1k\Omega) = 3k\Omega$$

$$V_{out} = 12V \times \left(\frac{3k\Omega}{7.3k\Omega} \right) = 4.93V$$

The voltage threshold of the second comparator can be calculated as follows

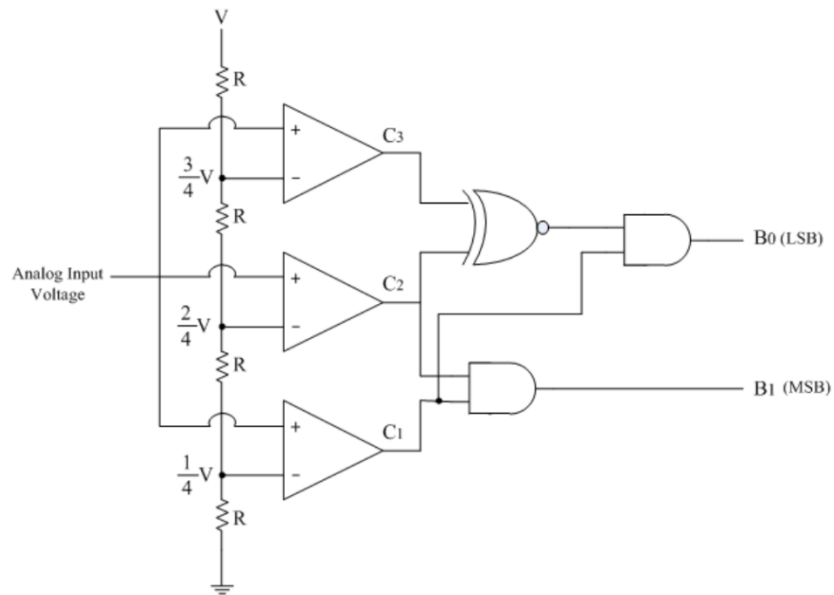
$$R_{below} = (1k\Omega + 1k\Omega) = 2k\Omega$$

$$V_{out} = 12V \times \left(\frac{2k\Omega}{7.3k\Omega} \right) = 3.29V$$

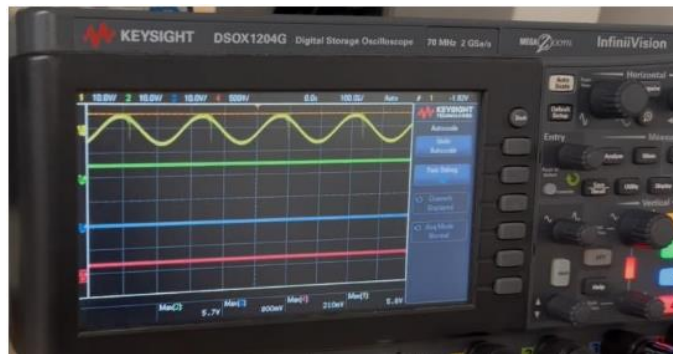
Calculating the last comparators voltage threshold

$$R_{below} = 1k\Omega$$

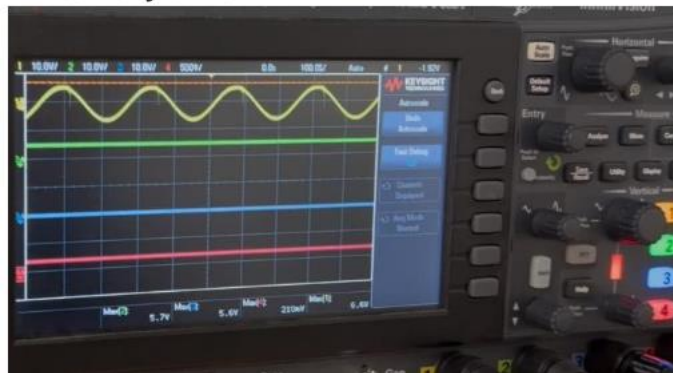
$$V_{out} = 12V \times \left(\frac{1k\Omega}{7.3k\Omega} \right) = 1.64V$$



Appendix Figure 4.8.1: Reference schematic of 2-bit flash ADC



Appendix Figure 4.8.5: Oscilloscope reading of ADC bit 001



Appendix Figure 4.8.6: Oscilloscope reading of bit 011



Appendix Figure 4.8.7: Oscilloscope reading of bit 111

Appendix 5 — PCB Design

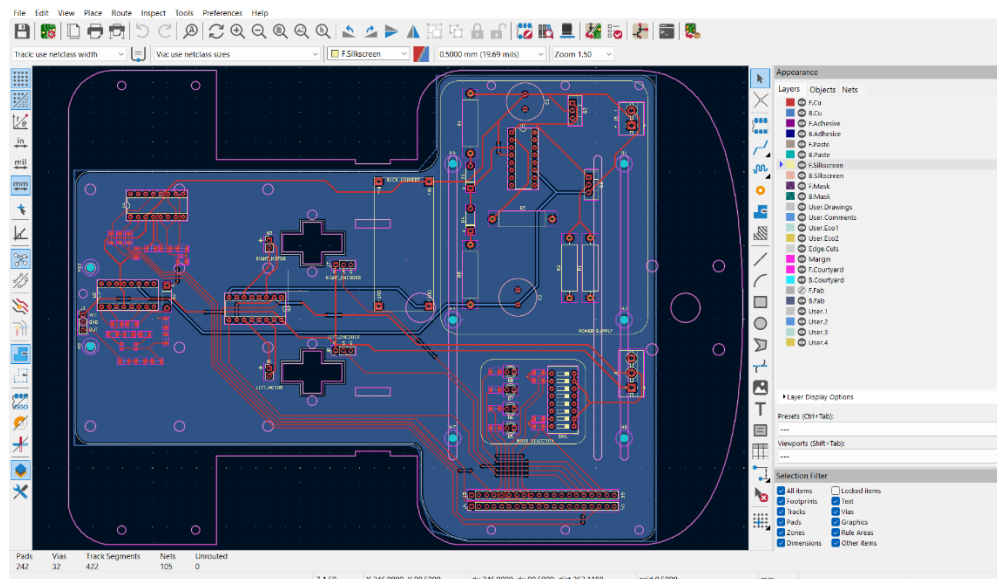


Figure 5.1.1: Screenshot of KICAD UI with final PCB design

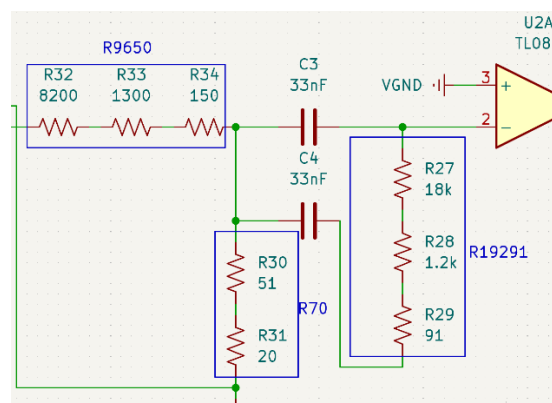


Figure 5.2.2: Close-up of realistic component values stacked to achieve desired resistance

Designator	Footprint	Quantity	Value	LCSC Part #
C1, C2	C_Radial_D12.5mm_H20.0mm_P5.00mm	2	200uF	C443147
C3, C4	C_0603	2	33nF	C113810
C5	C_0402	1	0.1u	C6376108
D1, D4	D_DO-41_SOD81_P7.62mm_Horizontal	2	1N4007	C19357
D2	D_DO-41_SOD81_P7.62mm_Horizontal	1	1N4007W	C19357
D5, D6, D7	LED_D3.0mm	4	LED	C330774
J1	PinHeader_1x03_P2.54mm_Vertical	1	Mic	C2932698
J3, J4	PinHeader_1x20_P2.54mm_Vertical	2	Conn_01x20_Pin	C50981
J5	TerminalBlock_MaiXu_MX126-5.0-02P_1x02_P5.00mm	1	Screw_Terminal_01x02	C8404
J6	PinHeader_1x03_P2.54mm_Vertical	1	Left Encoder	C2932698
J7	PinHeader_1x03_P2.54mm_Vertical	1	Right Encoder	C2932698
J8	TerminalBlock_MaiXu_MX126-5.0-03P_1x03_P5.00mm	1	Screw_Terminal_01x03	C5162336
M1	PinHeader_1x02_P2.54mm_Vertical	1	LEFT_MOTOR	C2905434
M2	PinHeader_1x02_P2.54mm_Vertical	1	RIGHT_MOTOR	C2905434
Q1	TO-220-3_Vertical	1	IRF4905	C2564
Q2	TO-220-3_Vertical	1	IRFZ34N	C21066
R1, R2	R_Axial_DIN0516_L15.5mm_D5.0mm_P20.32mm_Horizontal	2		10 C58716
R10	R_1206	1	5.1k	C26033
R11, R12, R13	R_1206	5	1k	C1469
R15	R_1206	1	4.3k	C18009
R16, R17, R18	R_1206	7	10k	C1489
R23, R24, R25	R_1206	4		470 C4502
R27	R_1206	1	18k	C17921
R28	R_1206	1	1.2k	C17891
R29	R_1206	1		91 C1448
R3, R4, R5	R_Axial_DIN0516_L15.5mm_D5.0mm_P20.32mm_Horizontal	3	100K	C57936
R30	R_1206	1		51 C18019
R31	R_1206	1		20 C17955
R32	R_1206	1		8200 C1488
R33	R_1206	1		1300 C1471
R34	R_1206	1		150 C17917
SW1	SW_DIP_SPSTx08_Slide_9.78x22.5mm_W7.62mm_P2.54mm	1	SW_DIP_x08	C19078528
U1, U2	DIP-16_W7.62mm_Socket	2	TL084	C2326
U3	DIP-16_W7.62mm_Socket	1	SN754410NE	C2326
U4	DIP-16_W7.62mm_Socket	1	LM339	C2326
U5	DC-DC Buck Converter 45x20mm	1	~	

Table 5.3.2: Full list of components and corresponding footprints (Bill of Materials)

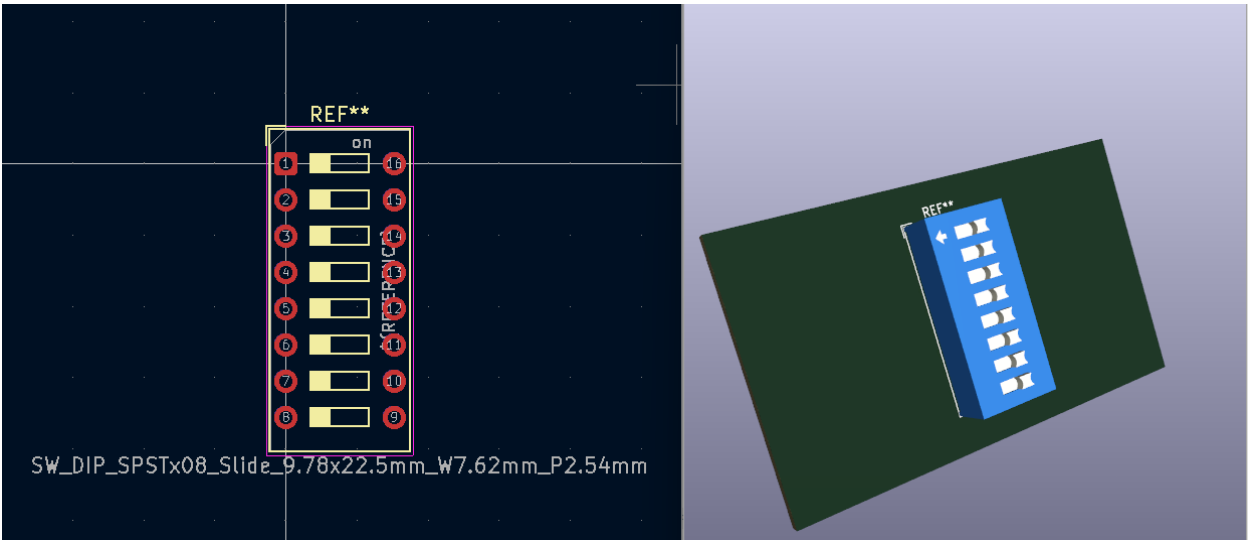
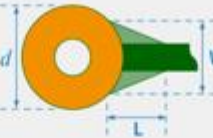


Figure 5.3.3: Footprint example of 8 pin DIP switch including 3D model

Copper			
	Minimum clearance:	0.1	mm
	Minimum track width:	0.16	mm
	Minimum connection width:	0	mm
	Minimum annular width:	0.2	mm
	Minimum via diameter:	0.25	mm
	Copper to hole clearance:	0.25	mm
	Copper to edge clearance:	0.5	mm
Holes			
	Minimum through hole:	0.15	mm
	Hole to hole clearance:	0.2	mm
uVias			
	Minimum uVia diameter:	0.25	mm
	Minimum uVia hole:	0.15	mm
Silkscreen			
	Minimum item clearance:	0	mm
	Minimum text height:	0.8	mm
	Minimum text thickness:	0.08	mm

Default Properties for Round Shapes



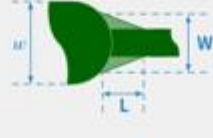
Best length (L): 50 % (d)
Maximum length (L): 1 mm
Best width (W): 100 % (d)
Maximum width (W): 2 mm
☐ Curved edges

Default Properties for Rectangular Shapes



Best length (L): 50 % (w)
Maximum length (L): 1 mm
Best width (W): 100 % (w)
Maximum width (W): 2 mm
☐ Curved edges

Properties for Track-to-Track Teardrops



Best length (L): 50 % (w)
Maximum length (L): 1 mm
Best width (W): 100 % (w)
Maximum width (W): 2 mm
☐ Curved edges

Table 5.3.4: Design constraints for JLCPCB in KiCAD’s board setup

J@LCPCB

Why JLCPCB?

Capabilities

Support

Resources

Q

Order now

My file

joshremhob

1

AudioCar2

PCB

Bill of Materials

Component Placements

Quote & Order

Automatically saved, last updated on 4 April, 12:35

Charge Details

PCB Price

\$15.70

Engineering fee:

\$4.00

Via Covering:

\$0.00

Board:

\$11.70

Economic PCBA Price

\$82.48

Setup Fee:

\$8.00

Stencil:

\$1.50

Panel:

\$0.00

Large Size:

\$0.00

Components(28 items):

\$13.62

Extended components fee:

\$42.00

SMT Assembly

\$1.05

Hand-soldering labor fee:

\$3.50

Manual Assembly

\$12.81

Build Time:

PCB: 24 hours

\$0.00

Assembly: 2 - 3 days

\$0.00

Total Price:

\$98.18

Weight

2.26kg

Product Description

Reserch\Education\DIY\Entertainment / D...

SAVE TO CART

Table 5.4.2: Final quote of board fabrication, parts acquisition, and final assembly

References

- [1] H. Zumbahlen, "MT-218 (rev. A)," Multiple Feedback Band-Pass Design Example, <https://www.analog.com/media/en/training-seminars/tutorials/mt-218.pdf> (accessed Apr. 23, 2025)
- [2] J. M. Fiore, *Operational Amplifiers and Linear Integrated Circuits: Theory and Application*. Chapter 11, section 7. Jaico Publishing House, 2006
- [3] "Flash ADC," All About Circuits, <https://www.allaboutcircuits.com/textbook/digital/chpt-13/flash-adc/> (accessed Apr. 23, 2025)..
- [4] Texas Instruments, "LM339A Datasheet," <https://www.ti.com/lit/ds/symlink/lm339a.pdf> (accessed Apr. 23, 2025).
- [5] "Flash Type ADC," Electronics-Tutorial.net, <https://www.electronics-tutorial.net/analog-integrated-circuits/data-converters/flash-type-adc/> (accessed Apr. 23, 2025).
- [6] "Virtual Ground Circuits," tangentsoft.com, <https://tangentsoft.com/elec/vgrounds.html> (accessed Apr. 23, 2025).
- [7] T. Agarwal, "Push-pull amplifier - circuit diagram, working and application." (2020, June 11) <https://www.elprocus.com/what-is-a-push-pull-amplifier-circuit-diagram-and-its-working-principle/> (accessed Apr. 23, 2025).
- [8] Texas Instruments, *L293, L293D Quadruple Half-H Drivers Datasheet*, <https://www.ti.com/lit/ds/symlink/l293.pdf> (accessed Apr. 23, 2025).